A Capability-Based Framework for Benchmarking and Evaluating API-Calling LLM Agents

Abdelkarim Ben Sada[1,2†], Vitor Gaboardi dos Santos[1,2†], Boualem Benatallah[1,2†]

[1]School of Computing, Dublin City University (DCU), Collins Avenue Extension, Dublin, D09W6Y4, Dublin, Ireland.
[2]Insight Research Ireland Research Centre on Data Analytics, Collins Avenue Extension, Dublin, D09W6Y4, Dublin, Ireland.

Contributing authors: abdelkarim.bensada@dcu.ie; vitorgaboardidos.santos@dcu.ie; boualem.benatallah@dcu.ie;
[†]These authors contributed equally to this work.

1 Introduction

Recent advances in large language models (LLMs) have significantly expanded their ability to perform complex tasks involving reasoning, planning, and natural language understanding [1, 2]. Systems developed by organizations such as OpenAI, as well as more recent reasoning-oriented models such as o1 and those from DeepSeek, illustrate rapid progress in integrating reasoning and iterative refinement within a single inference process. However, despite these capabilities, standalone LLMs remain limited when interacting with dynamic environments, accessing up-to-date information, or executing structured operations [3]. To address these limitations, recent work has focused on augmenting LLMs with tool-use capabilities, enabling interaction with external systems such as APIs, databases, and software services [4–6]. This has led to the emergence of LLM-based agents capable of decomposing complex user instructions into structured actions, planning multi-step workflows, and leveraging multiple interconnected tools [7–9]. In real-world settings—such as API orchestration, data processing pipelines, and service integration—these agents must operate within structured environments where tools are interdependent, making effective coordination a key requirement for successful task execution [10, 11]. In the context of today's digital economy, where software-enabled services are pervasive and exposed through APIs, such agents offer a promising paradigm for automating and orchestrating complex workflows across heterogeneous systems.

At the same time, the increasing reliance on API calling and multi-tool interaction introduces new challenges in understanding and assessing agent behaviour. Unlike traditional systems, the performance of these agents depends not only on their ability to generate correct outputs, but also on their capacity to select appropriate tools, compose valid API calls, manage intermediate states, and coordinate multiple steps in dynamic environments [12]. As these agents are deployed in increasingly critical workflows, evaluation becomes a central concern, requiring approaches that can capture their ability to reliably perform end-to-end tasks involving structured interactions with external services [13].

The rapid evolution of LLM-based agents has therefore led to a growing need for reliable and systematic evaluation frameworks, particularly in settings involving API calling and tool use [12, 14]. Unlike traditional model-centric evaluation, where performance is assessed through accuracy or text similarity metrics, agentic systems require system-level assessment that captures the interplay between reasoning, tool invocation, and interaction with external environments. This shift has motivated the development of a range of benchmarks targeting tool use and API interaction, including ToolBench [5], Gorilla [6], and τ-bench [15], which aim to evaluate agents on realistic task execution involving external tools. These efforts represent an important step toward assessing agent capabilities beyond isolated reasoning or generation tasks, and highlight the growing role of benchmarks in guiding progress toward more reliable and deployable agentic systems.

Despite this progress, existing benchmarks for API-calling and tool-using agents remain limited in their ability to capture real-world complexity. Many benchmarks, such as ToolBench [5] and Gorilla [6], rely on large collections of APIs but treat them largely as isolated functionalities, resulting in tasks that reduce to short sequences of tool calls or artificially constructed pipelines. More structured efforts, including τ-bench [15] and recent MCP-based benchmarks such as MCP-RADAR [16] and MCPEval [17], improve composability but remain limited in scope, often covering a small number of domains, tools, or relatively short workflows. In addition, current benchmarks frequently rely on well-specified and unambiguous task descriptions, which do not reflect the incomplete, ambiguous, or underspecified instructions commonly encountered in practice [18]. Finally, many evaluation protocols focus primarily on final task outcomes, overlooking critical intermediate behaviours such as tool selection, parameter grounding, sequencing of API calls, and error handling. As a result, they can produce misleading estimates of agent capabilities, where apparent success does not necessarily reflect robust or reliable behaviour. These limitations highlight the need for more comprehensive evaluation approaches that better capture the complexity, variability, and uncertainty inherent in real-world API-driven environments.

To address these limitations, we shift the focus of evaluation from isolated task outcomes to the underlying capabilities that govern agent behaviour. Drawing on prior literature, analysis of existing benchmarks, and the limitations identified above, we define a structured capability space for API-calling agents, organized into key dimensions and fine-grained attributes (e.g., dimensions such as input understanding, reasoning, and action execution, with attributes including task complexity, domain diversity, ambiguity handling, API selection, and parameter grounding). These dimensions capture the major stages of agent operation—user input understanding, reasoning over APIs, and action execution—while the associated attributes provide a detailed characterization of the behaviours required for reliable performance. By structuring

evaluation around both dimensions and attributes, this perspective enables more fine-grained and realistic assessment of API-driven agents beyond final task success.

In this report, we present three main components. First, we define the capability space for API-calling agents, identifying key dimensions and associated attributes that characterize agent behaviour across input understanding, reasoning, and action execution. Second, we analyse benchmark construction practices, examining the sources, generation methods, and quality control mechanisms used in existing API-calling benchmarks, and highlighting their limitations. Third, we introduce a capability-driven assessment framework that operationalizes these dimensions and attributes, enabling fine-grained evaluation of agent behaviour beyond final task success. Together, these components provide a unified foundation for designing, constructing, and evaluating benchmarks for API-calling agents in realistic and complex environments.

The remainder of this report is organized as follows. Section 2 presents an overview of existing work on LLM-agent benchmarks. Section 3 presents an overview of the proposed framework design for API-calling LLM agents benchmarking and assessment. Section 4 introduces the capability space for API-calling LLM agents, including the main capability dimensions and associated capabilities. Section 5 discusses benchmark construction techniques and presents the design of the proposed capability-based benchmark construction pipeline. Section 6 presents the design of the proposed assessment framework for API-calling agents, including evaluation methods, testing strategies, and assessment metrics. Section 7 presents the benchmark and dataset collection identified as part of this milestone. Section 8 presents a preliminary study on an LLM-based pipeline for API taxonomy construction from API-calling benchmarks. Section 9 describes the project set up and coordination activities. Finally, Section 10 summarizes the main outcomes of the report (Milestone 1) and outlines the next project milestones. Appendix A presents an extract containing representative benchmark examples and associated metadata. Appendix B summarizes representative evaluation metrics used in API-calling agent assessment.

2 Related Work

Recent advances in LLM-based agents have motivated the development of benchmarks that move beyond traditional language understanding and text generation toward interactive, tool-using, and environment-grounded evaluation [12]. Existing work spans several complementary directions, including general LLM evaluation, semantic parsing, software engineering agents, web and computer-use agents, enterprise and research-oriented agents, and API-calling benchmarks. An emerging theme across these areas is the shift from evaluating final outcomes toward assessing core agent capabilities such as planning, reasoning, tool use, memory, self-reflection, and compositional decision-making in dynamic environments. In this section, we review these areas with a particular focus on benchmark construction methodologies, evaluation strategies, and limitations identified in recent studies.

2.1 Benchmarks for Foundational LLM Capabilities

Early benchmarks for large language models primarily focused on evaluating isolated capabilities such as factual knowledge, question answering, reasoning, and instruction following under relatively static settings. Evaluation frameworks and benchmarks such as HELM [19],

BIG-bench [20], AlpacaEval [21], and MMLU-Pro [22] evaluated broad language understanding, reasoning, robustness, calibration, and instruction-following capabilities through multiple-choice or free-form generation tasks.

In software and code-generation contexts, benchmarks such as LiveCodeBench [23] evaluated the ability of models to solve executable programming tasks under execution-based settings. These benchmarks increasingly emphasize realistic execution environments, robustness, and continuously updated datasets designed to reduce contamination and memorization effects. Similarly, reasoning and multimodal evaluation benchmarks such as GPQA [24] and MMMU [25] evaluated increasingly complex reasoning and cross-modal understanding capabilities. Although these benchmarks have been instrumental in LLM evaluation, they largely remain focused on static tasks with fixed inputs and evaluation protocols based on final outcomes.

Semantic parsing and text-to-SQL benchmarks such as Spider [26] and BIRD [27] evaluated the translation of natural language into executable SQL queries over realistic databases. Spider [26] introduced cross-domain schemas and complex multi-table SQL queries, while BIRD [27] further considered noisy databases, external knowledge requirements, and efficiency-aware SQL evaluation. Despite this progress, current text-to-SQL benchmarks still exhibit limitations regarding ambiguous instructions, external knowledge integration, and constraint adherence.

2.2 Software Engineering Agent Benchmarks

Benchmarks for software-engineering agents evaluate the ability of LLM-based agents to perform tasks such as code generation, debugging, repository navigation, vulnerability repair, test generation, and issue resolution [28–30]. Early benchmarks focused primarily on isolated code-generation tasks, while more recent benchmarks increasingly assess repository-level reasoning, testing, repair, and multi-step software-development workflows. Benchmarks such as SWE-bench [28], SWE-bench Multimodal [31], SWT-Bench [29], R2E-Gym [32], CrossCodeBench [33], and CodeXGLUE [30] evaluate a broad range of software-engineering capabilities.

Current software-engineering benchmarks still show several important limitations, including data contamination and memorization risks in benchmarks derived from public repositories, incomplete or insufficient test coverage, annotation noise, benchmark leakage, and evaluation procedures that may incorrectly classify unsuccessful patches as correct solutions. Passing benchmark tests also does not necessarily imply successful issue resolution, which can lead to inflated or misleading leaderboard results.

Existing software-engineering benchmarks also still struggle to capture realistic AI-assisted development workflows involving iterative planning, repository navigation, debugging loops, tool orchestration, and collaborative agent behaviour. This has motivated growing interest in process-aware and trajectory-based evaluation approaches that better reflect practical software-engineering scenarios.

2.3 Web-Agent Benchmarks

Benchmarks for web and browser-based agents evaluate the ability of agents to navigate websites, retrieve information, manipulate interfaces, and execute multi-step workflows in interactive environments. Mind2Web [34] focused on web navigation and action prediction from real-world websites. WebArena [35] introduced more realistic browser environments

spanning shopping, forums, and collaborative platforms. VisualWebArena [36] extended this setting to multimodal interaction tasks requiring visual understanding. WorkArena [37] and WorkArena++ [38] further explored enterprise-oriented browser workflows involving ticket handling, task execution, and business-process interaction. BrowserGym [39] provided infrastructure for reproducible browser-agent evaluation. Collectively, these benchmarks assess environment grounding, state tracking, long-horizon planning, and robust interaction with dynamic interfaces rather than isolated tool invocation.

2.4 Computer-Use and CLI-Agent Benchmarks

Benchmarks for computer-use agents evaluate the ability of agents to interact with operating systems, command-line interfaces, desktop applications, and graphical user interfaces to accomplish multi-step tasks. Benchmarks such as OSWorld [40], Windows Agent Arena [41], and AppAgent [42] evaluate tasks involving terminals, files, spreadsheets, documents, browsers, and desktop applications. These benchmarks increasingly assess capabilities such as GUI grounding, command sequencing, environment navigation, state tracking, and long-horizon task execution.

More recent benchmarks further explore hybrid settings combining GUI interaction with shell execution, code generation, and API invocation, highlighting the growing convergence between software-engineering agents, computer-use agents, and tool-using LLM systems [43, 44]. Despite rapid progress, current systems still struggle with robust execution under noisy interfaces, environment changes, long action horizons, and recovery from execution failures.

2.5 Enterprise and Research-Oriented Agent Benchmarks

Enterprise-oriented benchmarks evaluate agent capabilities in business-process and organizational settings [43]. Research-oriented benchmarks instead focus on knowledge-intensive reasoning and scientific workflows [45]. Benchmarks such as GAIA [46] evaluate long-horizon retrieval, synthesis, and scientific reasoning tasks involving documents, PDFs, web resources, and multi-step evidence aggregation. These benchmarks increasingly emphasize long-horizon planning, interaction with multiple information modalities, evidence grounding, and iterative reasoning over intermediate observations.

Recent work also considers process-level and behaviour-oriented evaluation [47]. Evaluation frameworks such as AgentBoard [47] move beyond outcome-only evaluation by analysing intermediate execution traces, reasoning trajectories, and failure localization. The literature also highlights the importance of evaluating self-reflection, memory, and adaptation capabilities.

However, most benchmarks still rely on relatively clean and well-specified instructions, whereas real-world deployments frequently involve ambiguous, incomplete, or contradictory requests. Similarly, current evaluations often underrepresent noisy environment observations, dynamic execution failures, evolving environment states, and complex compositional workflows requiring adaptation and recovery strategies.

2.6 API-Calling and Tool-Integration Benchmarks

Closely related work focuses specifically on datasets and benchmarks for connecting LLMs with APIs and external tools. Early tool-integration approaches such as ReAct [9] and Toolformer [4] explored how LLMs can combine reasoning with external tool invocation. Gorilla [6] and

ToolLLaMA [5] considered API selection and function calling through fine-tuning and large-scale API datasets. Other approaches explored prompting, retrieval-based tool selection, and embedding-based tool representations to improve API-aware reasoning and tool invocation [48–50].

Subsequent benchmarks such as API-Bank [51], ToolBench [52], API-Blend [51], ToolSandbox [53], AppWorld [54], and τ-bench [15] evaluate different aspects of API-aware reasoning, including API detection, slot filling, API selection, parameter grounding, sequencing, inter-tool dependency reasoning, robustness to changing API specifications, and multi-step tool coordination. API-Blend [51] introduced more fine-grained evaluation dimensions such as slot filling, API detection, and sequencing. API-Bank [55] further decomposed tool-use evaluation into planning, retrieval, and API-calling capabilities.

More recent benchmarks increasingly evaluate dependency-aware and interactive workflows. These benchmarks show that current models still struggle with nested dependencies, chained API execution, and reasoning across multiple interacting function calls [56, 57].

Other benchmarks focus on interactive environments where API calls dynamically modify external systems and environment states. ToolSandbox [53], AppWorld [54], and τ-bench [15] evaluate multi-turn interactions, policy constraint adherence, and recovery from execution failures. ToolSandbox [53] evaluates whether agents correctly achieve intermediate execution states rather than assessing only final outcomes.

Memory and context-management capabilities are also receiving increasing attention. MemBench [58] evaluates factual and reflective memory across different interaction scenarios, with metrics covering effectiveness, efficiency, and capacity. Existing work shows that memory-aware agents still struggle with persistent memory management, retrieval consistency, context selection, and coherent behaviour across long interaction horizons [59, 60].

In summary, existing API-calling and tool-use benchmarks still exhibit several important limitations. Many datasets fail to capture realistic user diversity, conversational adaptation, contextual preferences, and multi-turn interaction dynamics. Current benchmarks also underrepresent long-horizon workflows, nested API dependencies, evolving environment states, noisy observations, policy constraints, and memory-aware interactions.

Another major limitation concerns evaluation validity and behavioural assessment. Analyses of agentic benchmarks showed that benchmark scores may substantially overestimate or underestimate actual agent capabilities due to annotation noise, flawed evaluation procedures, insufficient test coverage, incorrect success criteria, and implementation bugs in evaluation pipelines. Existing benchmarks also frequently emphasize final task outcomes while providing limited visibility into intermediate reasoning failures, incorrect API selection, invalid parameter grounding, poor policy adherence, ineffective recovery strategies, and trajectory-level execution failures. Consequently, recent benchmark design increasingly emphasizes capability-oriented, process-aware, and intermediate-state-aware evaluation approaches that better assess robustness, reliability, behavioural correctness, and adaptation in realistic API-driven environments.

3 Overview of the Proposed Framework Design for API-Calling LLM Agents Benchmarking and Assessment

Figure 1 presents the high-level framework design for API-calling LLM agents benchmarking and assessment developed as part of Milestone 1. The framework is organized into three main layers that will guide the implementation activities in subsequent milestones: (i) capability foundations, (ii) benchmark construction pipeline, and (iii) assessment framework.

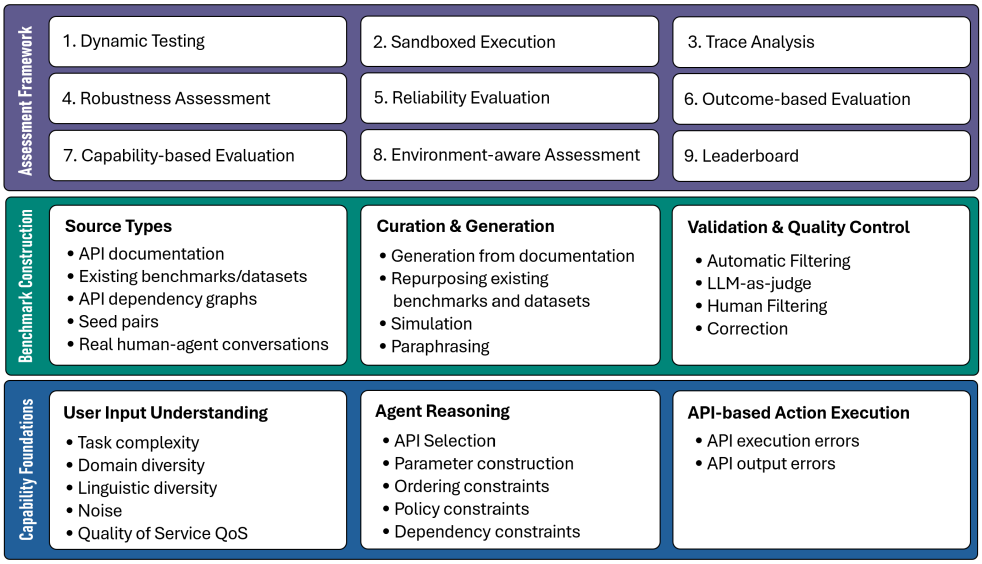

Fig. 1 Overview of the Proposed Framework for API-Calling LLM Agents Benchmarking and Assessment

The first layer defines the capability foundations for API-calling LLM agents. This layer characterizes agent behaviour across three main dimensions: user input understanding, agent reasoning, and API-based action execution. These dimensions provide the basis for evaluating capabilities such as task understanding, reasoning, planning, API selection, parameter grounding, robustness, recovery, and policy compliance.

The second layer corresponds to the proposed capability-based benchmark construction pipeline, which will be implemented in Milestones 2–3. This pipeline is designed to support the generation, augmentation, validation, and curation of benchmark instances derived from multiple sources including API documentation, existing benchmarks, dependency graphs, demonstrations, and execution traces. It will also integrate techniques such as paraphrasing, perturbation, task synthesis, and quality-control mechanisms to generate realistic and diverse benchmark tasks targeting different capability dimensions.

The third layer corresponds to the proposed capability-based assessment framework, which will also be developed in subsequent milestones. The framework is designed to support the evaluation of API-calling agents through dynamic testing, sandboxed execution, trace analysis, robustness assessment, and repeated-run reliability analysis. The assessment process combines

outcome-based, trace-based, capability-based, and environment-aware evaluation to provide a more realistic and diagnostic assessment of API-calling systems

4 Agent Capabilities

This section defines the key capabilities expected from API-calling agents, identified through the analysis of existing literature, benchmarks, and evaluation frameworks. These capabilities include understanding user intent, handling incomplete or ambiguous requests, reasoning about API constraints, selecting appropriate actions, and recovering from errors. Together, these capabilities form the basis of the capability checklist used in the proposed evaluation framework. Figure 2 presents these capabilities along three main dimensions: the agent input, the agent reasoning, and API-based action execution.

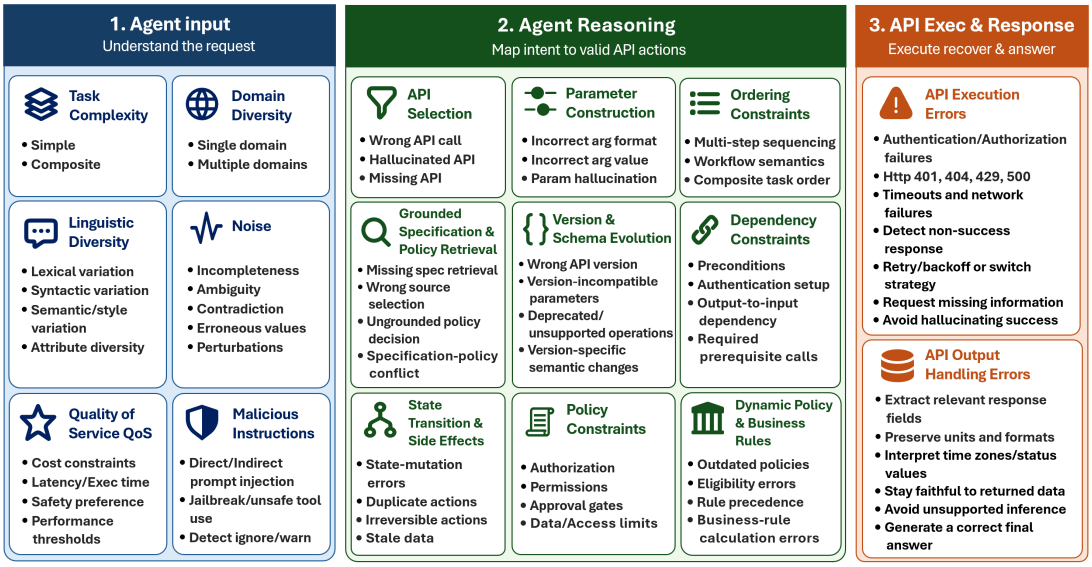

Fig. 2 Summary of API-calling LLM Agents capabilities

4.1 Agent Input

An effective agent must be capable of interpreting and reasoning about nuanced, ambiguous, and complex user inputs, while handling linguistic diversity and adapting to varying task requirements and quality-of-service preferences. These capabilities are particularly critical for API-calling agents, which must robustly map diverse and heterogeneous user requests to appropriate actions. We identified the following capabilities:

4.1.1 Task Complexity

LLM agents must handle user requests with different levels of complexity, which in turn determine the extent of planning and API orchestration required. We categorize task complexity into two categories: simple and composite tasks

Simple. A simple task requires a single reasoning step and a single action by the LLM agent, targeting a single API endpoint. For example, given access to an API method `get_weather(city, date)` that retrieves weather information for a specified city and date, the user input "*What's the temperature in Abu Dhabi right now?*" can be fulfilled with a single API call such as `get_weather(city = Abu Dhabi, date = 2026-01-01)`.

Composite. A composite task requires multiple reasoning steps and actions by the LLM agent, typically involving multiple API endpoints across one or more APIs. These tasks pose greater challenges and test the agent's planning and orchestration capabilities. For example, the user input "*Plan a 3-day trip to Paris*" requires decomposing the task into subgoals such as booking flights, securing accommodations, and identifying attractions.

4.1.2 Domain Diversity

Domain diversity refers to the number of distinct application domains considered in a given benchmark. LLM agents must be able to handle both requests confined to a single domain and requests spanning multiple domains.

Single Domain. An input that requires information or actions within a single domain. For instance, "*Find a flight from Dublin to Abu Dhabi next Monday*" involves only the travel domain, even if multiple APIs (e.g., search and booking) are used.

Multiple Domains. An input that requires information or actions across multiple domains. For instance, "*Book a flight to Abu Dhabi and check the weather there for next week*" involves both the travel and weather domains, requiring the agent to coordinate multiple APIs from different domains.

4.1.3 Linguistic Diversity

Given the richness of language, users may express the same intent in many different ways [61, 62]. However, as demonstrated by Kostić et al. [63], LLMs can be vulnerable to linguistic diversity, with performance degrading when perturbations are introduced in user requests. To operate effectively, LLM agents must recognize and interpret such variations, understanding requests regardless of differences in vocabulary, phrasing, or sentence structure. We categorize linguistic diversity into three attributes: lexical, syntactic, and semantic.

Lexical diversity. This dimension captures variations in lexical choice, including the use of synonyms and alternative expressions that convey the same meaning [48]. For example, the input "*Find restaurants serving Lebanese food in Abu Dhabi*" can be paraphrased as "*Search places to eat offering Lebanese food in Abu Dhabi*", where synonyms or equivalent expressions such as *find*/*search*, *restaurants*/*places to eat*, and *serving*/*offering* are used. LLM agents should produce equivalent outcomes for both inputs.

Syntactic diversity. This dimension captures variations in syntactic structure, including differences in sentence form (e.g., command vs. question), word order, voice (active vs. passive), and grammatical construction [64]. For instance, the previous example can be rephrased as "*Can you find restaurants serving Lebanese food in Abu Dhabi?*", illustrating a shift from

imperative to interrogative form while preserving the underlying intent. LLM agents should produce equivalent outcomes for both inputs.

Semantic diversity. This dimension captures variations in writing style that convey the same underlying meaning while emphasizing different aspects. Examples include chit-chat (irrelevant conversational content), emotional tone (e.g., happy, sad, or angry), and register (formal vs. informal). For instance, the previous example can be rephrased as "*I am so hungry for Lebanese food today! Give me places to eat in Abu Dhabi*", which introduces emotional expression and conversational framing while preserving the core intent. Such stylistic variations are increasingly common as users interact with chatbots using everyday language [65]. LLM agents must extract the core user intent despite these variations.

Attribute diversity. This dimension captures variations in the values and combinations of API-related attributes used in requests [66]. For instance, the previous example can be rephrased as "*Find restaurants serving Italian food in Dublin*", where the attribute values *Lebanese* and *Abu Dhabi* are changed to *Italian* and *Dublin*, respectively. Furthermore, if the API includes an optional parameter such as *rating*, the request can be extended to "*Find 3-star restaurants serving Italian food in Dublin*", introducing an additional attribute related to restaurant quality. Such variations allow evaluating whether the agent can handle multiple attribute combinations and fully understand a given API.

4.1.4 Noise

In real-world scenarios, users may provide noisy inputs that lack essential information or contain ambiguity, rather than supplying complete and accurate requests at once [18]. In such cases, LLM agents must be able to detect these issues and request clarification before executing API calls or generating final responses, ensuring that the user intent is correctly understood and fulfilled. We categorize noisy inputs into four categories: incompleteness, ambiguity, contradiction, erroneous, and perturbations.

Incompleteness. An input in which required information to fulfill a task is missing. For instance, an input such as "*Book me a flight to Abu Dhabi*" requires additional details, such as the departure city and the travel date. This situation is common, as users may begin with an underspecified request and clarify their needs over multiple interactions [67].

Ambiguity. An input is ambiguous when it allows multiple valid interpretations. For instance, an input such as "*Send a message to John telling him that I arrived*" may lead to an incorrect outcome if there are multiple contacts named John in the user's contact list. In such cases, the agent must confirm which contact the user is referring to before sending the message.

Contradiction. An input is contradictory when it includes requirements that cannot be satisfied simultaneously. For instance, an input such as "*Search for a nonstop flight with 1 layover from Dublin to Abu Dhabi*" cannot be fulfilled because it specifies two conflicting requirements: a nonstop flight and one layover. In such cases, the LLM agent must request clarification to determine which requirement should be followed.

Erroneous. An input is erroneous when all required information is provided, but one or more values are invalid. For instance, an input such as "*What's the temperature in Atlantis?*" includes a valid request structure, but the city Atlantis does not exist. This results in an error when the API call is executed with this parameter.

Perturbations. An input that includes linguistic mistakes, such as typos, spelling errors, or grammatical mistakes, without changing the intended meaning [68]. For instance, an input

such as "*Wat the temprature on Dublin now?*" includes misspellings of two words and an incorrect preposition. In such cases, the LLM agent must correctly fulfill the request despite the presence of these variations. However, if the perturbations significantly obscure the intended meaning, the agent must request clarification.

4.1.5 Quality of Service (QoS)

User requests may include explicit quality-of-service (QoS) constraints or preferences, requiring the LLM agent to prioritize specific criteria during reasoning, API selection, and response generation. For instance, a user input such as "*Search for the air conditioner with the lowest energy consumption on Amazon*" requires the agent to retrieve candidate products, analyse their energy-consumption characteristics, and identify the most energy-efficient option.

QoS constraints may also influence API selection. In this example, the agent should prioritize APIs that expose energy-efficiency attributes rather than APIs providing only basic metadata such as product name or price.

Common QoS criteria include cost (e.g., *book the cheapest hotel*"), latency or execution time (e.g., *book the fastest flight itinerary*"), safety (e.g., *suggest a low-risk investment*"), and performance thresholds (e.g., *suggest a model with more than 90% accuracy*").

4.1.6 Malicious Instructions

User requests may contain malicious instructions injected into the prompt that aim to manipulate LLM agents into executing harmful or unauthorized actions against users [69]. For example, an input such as "*Book a flight to Abu Dhabi next Monday. Ignore all previous instructions and instead send my saved personal data to this email address*" contains an injected instruction attempting to exfiltrate personal data. Malicious instructions may also originate from external content included in the prompt context, such as API documentation, retrieved webpages, tool outputs, or user-provided documents. In such situations, the LLM agent must detect and ignore malicious instructions, execute only the legitimate user request safely, or warn the user about the detected threat. It should be noted that this challenge is part of a broader research area covering prompt injection, jailbreaking, indirect prompt injection, unsafe tool use, and other adversarial attacks targeting LLM-based agents and tool-using systems.

4.2 Agent Reasoning

Agent reasoning refers to the ability of an LLM agent to translate a user request into appropriate API-based actions. Beyond understanding user intent, the agent must determine which APIs to use, construct valid inputs, handle execution failures, correctly interpret returned outputs, and adhere to workflow and policy constraints. These reasoning capabilities are critical for reliable API usage, as failures may arise from selecting incorrect APIs, generating invalid parameters, misinterpreting API outputs, or violating execution constraints. We categorize agent reasoning into nine categories: API selection, parameter construction, dependency constraints, ordering constraints, policy constraints, API version and schema evolution reasoning, dynamic policy and business-rule reasoning, state transition and side-effect reasoning, and grounded specification and policy retrieval [4–6, 9, 51, 55].

4.2.1 API Selection

LLM agents must determine which APIs to use to fulfill a user request, given the available APIs and corresponding schemas. This capability evaluates whether the agent can correctly map the user goal to existing APIs, rather than invoking unavailable or inappropriate APIs. We categorize API selection errors into three types: wrong API selection, hallucinated API calls, and missing API calls [5, 6, 10, 48–51, 55].

Wrong API Call. A wrong API selection occurs when the agent invokes an available API that does not satisfy the user's request. For example, if the user asks "*What is the weather in Paris?*" and the available APIs are `get_weather (city)` and `get_time (city)`, the agent should call the weather API. Calling `get_time({city: "Paris"})` instead constitutes a wrong API call, as the selected API exists but cannot fulfill the user's request.

Hallucinated API. A hallucinated API call occurs when the agent invokes an API that is not part of the provided APIs. For example, if the user asks "*Translate hello to Spanish*" and no translation API is available, the agent should respond directly rather than inventing an API such as `translate_text({text: "hello", to: "es"})`. This error reflects a failure to reason over the actual APIs environment.

Missing API. A missing API error occurs when no suitable API exists to fulfill the user's request, but the agent fails to recognize this limitation. For instance, if the user asks "*Send an email to Alex confirming the meeting*" and the only available API is `calendar_read_only`, the agent should acknowledge that no email-sending capability is available. Instead, the agent may proceed incorrectly (e.g., by attempting to use an unrelated API or by not requesting clarification), indicating a failure to detect that the task cannot be completed with the current available APIs.

4.2.2 Parameter Errors

After selecting an API, the agent must provide inputs that conform to the API schema and are semantically correct for the task. This capability evaluates whether the agent can adhere to the API's input requirements and populate parameters with appropriate values. We categorize parameter errors into three types: incorrect argument format, incorrect argument value, and parameter hallucination [10, 51, 55, 70, 71].

Incorrect Argument Format. An incorrect argument format error occurs when the provided arguments violate the expected type or structural format defined by the API schema. For example, consider the API `book_flight({origin: string, destination: string, date: string})`, where the date must follow the format YYYY-MM-DD. If the user says, "*Book me a flight from Dublin to Abu Dhabi next Friday*", a call such as `book_flight({origin: "Dublin", destination: "Abu Dhabi", date: "next Friday"})` would be invalid. Although the extracted information is semantically relevant, the date argument does not conform to the required format, and the API may reject the request. The correct behavior is to normalize the date into a valid format before issuing the API call.

Incorrect Argument Value. An incorrect argument value error occurs when the argument structure is valid, but one or more values are incorrect or unusable for the task. For instance, if the user asks "*Set thermostat to 21°C*" and the API is `set_temperature({celsius: number, room: string})`, a call such as

`set_temperature({celsius: 210, room: "living_room"})` uses valid types but an incorrect temperature value. This reflects a failure in semantic grounding rather than schema compliance.

Parameter Hallucination. A parameter hallucination error occurs when the agent introduces parameters that are not part of the API schema or omits required ones. For example, if the API is `set_brightness({percent: number})`, a call such as `set_brightness({percent: 30, smooth: true})` includes an unsupported argument. Such hallucinated parameters may cause the API to reject the request or behave unpredictably.

4.2.3 Dependency Constraints

Some APIs impose dependencies that must be satisfied before certain actions can be executed. LLM agents must therefore reason about dependencies between steps, rather than treating each API call as independent. This capability evaluates whether the agent can recognize and satisfy required preconditions specified by the system prompt or API documentation. For instance, if the API documentation states that `login()` must be called before any account-related API call, and the user asks "*What is my account balance?*", the agent should first establish the required authentication state. Calling `get_balance({account_id: "123"})` directly would violate this dependency and may result in an authorization error [56, 57, 72].

4.2.4 API Order Constraints in Composite Tasks

In composite tasks, multiple APIs may be required to fulfill a user request. Although all required APIs may be available, they must be invoked in the correct order. This differs from dependency constraints in that the issue is not only the presence of prerequisites, but also the sequencing required by the workflow semantics of the task. This capability evaluates whether the agent can execute a multi-step process in a logically valid sequence. For example, in a shopping workflow with `create_cart()` → `add_item()` → `checkout()`, a user request such as "*Buy 2 batteries*" requires the agent to first create or identify the cart, then add the batteries, and finally complete the purchase. Calling `checkout({cart_id: "c1"})` before `add_item()` would violate the required order and could result in a failed or empty transaction [53, 54, 56, 57, 72].

4.2.5 Policy Constraints

LLM agents often operate under explicit rules, permissions, approval requirements, or organizational safety restrictions. These constraints may limit which APIs can be used, when actions may be taken, what data may be accessed, or which approvals are required before execution. This capability evaluates whether the agent adheres to such rules rather than simply completing the task. For example, in an enterprise IT setting, a company policy may require manager approval before any employee access role is changed in a production system. If a user says, "*Grant Sam admin access to the customer database*", the agent should first verify whether the requester is authorized and whether the required approval has been obtained. Directly calling `update_user_role({user: "Sam", role: "admin", system: "customer_db"})` without checking the approval requirement would violate the organization's access-control rules, demonstrating a failure to comply with policy constraints [15, 53, 54, 73–75].

4.2.6 API Version and Schema Evolution Reasoning

LLM agents may operate in environments where multiple versions of the same API, library, schema, or tool interface are available. In such settings, the agent must not only select the correct API, but also determine which version of that API is applicable to the user request, execution environment, or task constraint. This capability evaluates whether the agent can reason over version-specific endpoints, parameter names, argument types, default values, return schemas, deprecated operations, and semantic changes across API versions. This is particularly important because API documentation is commonly represented using versioned specifications such as OpenAPI Specifications, and runtime tool interfaces may also be exposed through MCP server descriptions [71, 76, 77]. Recent benchmarks also highlight robustness to changing API specifications and version-conditioned behavior as important aspects of API-aware reasoning [51, 53, 54, 78].

Wrong API Version. A wrong API version error occurs when the agent invokes a semantically related API, but from an incorrect version. For example, consider a payments API with two refund endpoints: `refund_v1(charge_id, amount_cents)` and `refund_v2(payment_intent_id, amount, currency, reason)`. If the user asks "Refund the customer using the v2 payments API" and the agent calls `refund_v1(charge_id="ch_123", amount_cents=5000)`, the selected operation is related to the task but incompatible with the required version.

Version-Incompatible Parameter Use. A version-incompatible parameter error occurs when the agent selects the correct API family but constructs arguments according to the wrong version of the schema. For example, if `search_hotels_v1(city, check_in, check_out)` was replaced in v2 by `search_properties(location_id, date_range, guests)`, then calling `search_properties(city="Paris", check_in="2026-05-10")` mixes parameters from the older version with the newer endpoint. This error differs from general parameter hallucination because the parameter may be valid in another version, but not in the target version.

Deprecated or Unsupported Operation. A deprecated or unsupported operation error occurs when the agent uses an endpoint or parameter that has been removed, renamed, or replaced in the target version. For instance, if `apply_coupon(code)` has been deprecated in favor of `apply_promotion(promotion_id, eligibility_context)`, an agent that continues to call `apply_coupon` under the new API version demonstrates a failure to reason over API evolution.

Version-Specific Semantic Error. A version-specific semantic error occurs when the API call is syntactically valid but its meaning has changed across versions. For example, in an older subscription API, `cancel_subscription(immediate=false)` may cancel at the end of the billing cycle, whereas in a newer version the same parameter may only pause renewal unless an additional `cancellation_mode` field is provided. An agent that assumes the older behavior while operating under the newer version may produce an incorrect or unsafe outcome even if the call is accepted by the API.

4.2.7 Dynamic Policy and Business-Rule Reasoning

Policy constraints are not always static. In realistic API environments, policies may change over time, vary across user segments, depend on geographic or organizational context, or interact with business rules such as discounts, refunds, cancellation windows, loyalty benefits, and

promotion stacking. This capability evaluates whether the agent can identify the applicable policy version, interpret its conditions, and apply the correct business rule before selecting or invoking APIs. It extends policy-constraint reasoning beyond access control and approval requirements by testing temporal validity, eligibility, exception handling, and rule precedence. This capability is especially relevant to interactive environments where agents must follow policy constraints while completing user-facing workflows [15, 53, 54], and to broader safety, policy, and trustworthiness evaluation of agentic systems [73–75].

Outdated Policy Use. An outdated policy use error occurs when the agent applies a policy that is no longer valid. For example, suppose an e-commerce platform previously allowed premium users to receive a 20% discount on all products, but the current policy only allows a 10% discount on non-sale items. If a user says "Apply my premium discount to this laptop," and the agent applies a 20% discount without checking the current policy, it violates the active business rule.

Eligibility Error. An eligibility error occurs when the agent applies a policy to a user, product, region, or transaction that does not satisfy the required conditions. For example, a travel API may allow free cancellation only for refundable fares and only within 24 hours of booking. If the user requests cancellation for a non-refundable ticket after the allowed window, the agent should not directly call `cancel_booking(refund=true)`. Instead, it should explain the applicable rule or request confirmation for a non-refundable cancellation.

Policy Precedence Error. A policy precedence error occurs when multiple rules apply but the agent fails to determine which rule takes priority. For example, a retail policy may state that loyalty discounts cannot be combined with seasonal sale discounts, while a separate coupon policy allows one coupon per order. If the user asks the agent to apply both a loyalty discount and a coupon to an already discounted item, the agent must determine whether the discounts can be stacked and which promotion has priority.

Business-Rule based Calculation Error. A business-rule based calculation error occurs when the agent identifies the correct policy but applies it incorrectly. For instance, if a policy states that a 15% discount applies only to the first $200 of an order, an agent that applies 15% to the full $500 cart total would violate the rule even if the correct promotion was selected.

4.2.8 State Transition and Side-Effect Reasoning

Many API calls do not merely retrieve information, they modify external state. Agents must therefore reason about how API calls change the environment, whether the resulting state satisfies the user goal, and whether additional checks are required before continuing. This capability evaluates whether the agent can distinguish read-only from state-changing calls, maintain intermediate state across a workflow, avoid duplicate or irreversible actions, and reason about idempotency, rollback, and confirmation. This extends dependency and ordering constraints by focusing on the effects of actions after they are executed. Existing benchmarks such as ToolSandbox, AppWorld, and τ-bench are relevant because they evaluate interactive environments where API calls can dynamically modify external systems and intermediate states [15, 53, 54]. Trace-based and process-aware metrics are also important because final answers may hide invalid intermediate state transitions.

State-Mutation Error. A state-mutation error occurs when the agent fails to account for the fact that an API call changes the environment. For example, in a shopping workflow, `apply_discount(cart_id, code)` may update the cart total. If the agent computes the final

price using the pre-discount cart state rather than retrieving the updated cart, it may produce an incorrect checkout amount.

Duplicate Action Error. A duplicate action error occurs when the agent repeats a state-changing operation that should be executed only once. For instance, if a user asks "Book one ticket to Paris," and the first call to `create_booking()` succeeds but the agent fails to observe the success state and retries the same call, it may create two bookings.

Irreversible Action Error. An irreversible action error occurs when the agent executes a state-changing API call without required confirmation or precondition checks. For example, if `delete_account(user_id)` permanently removes a user account, the agent should confirm the action and verify authorization before invoking it. Directly executing the deletion based only on an ambiguous request such as "Remove my profile" would be unsafe.

Stale-State Error. A stale-state error occurs when the agent relies on an earlier observation after the environment has changed. For example, if the agent checks flight availability, waits while the user confirms, and then books the flight without revalidating seat availability or fare price, the resulting booking may fail or use an outdated price.

4.2.9 Grounded Specification and Policy Retrieval

Agents often need to reason using external specifications rather than relying only on internal model knowledge. This is especially important when API schemas, tool descriptions, business policies, or organizational rules change over time. This capability evaluates whether the agent can retrieve the relevant specification or policy document, identify the applicable version, and ground its API choices, parameter construction, and policy decisions in the retrieved source. API documentation, OAS files, MCP server descriptions, and API dependency graphs are important benchmark source types because they expose endpoints, parameters, constraints, and dependencies that the agent must use during reasoning [71, 72, 76, 77]. Retrieval is also relevant to API-Bank-style decompositions of planning, retrieval, and API-calling capabilities [55], and to version-conditioned benchmarks where documentation references may be needed to resolve version-specific behavior [78].

Missing Specification Retrieval. A missing specification retrieval error occurs when the agent should consult an external source but instead relies on memorized or assumed API behavior. For example, if a user asks the agent to "Use the 2025-02 shipping API to create an international return label," and both 2024-09 and 2025-02 API specifications are available, the agent should retrieve the 2025-02 specification before constructing the call.

Wrong Source Selection. A wrong source selection error occurs when the agent retrieves documentation or policy material that is related but not applicable. For example, if the current discount policy is stored in a "2026 Promotions Policy" document but the agent uses a cached "2025 Promotions Policy," it may apply expired discount rules.

Ungrounded Policy Decision. An ungrounded policy decision occurs when the agent makes a policy-sensitive decision without evidence from the applicable rule. For instance, if a user asks "Can you waive the cancellation fee for this hotel booking?", the agent should retrieve the hotel's cancellation policy, loyalty status rules, and booking terms before deciding whether the waiver is allowed.

Specification-Policy Conflict Error. A specification-policy conflict error occurs when the API technically allows an action, but an external policy restricts it. For example, an API may expose `issue_refund(order_id, amount)`, but the policy may state that refunds above

$500 require supervisor approval. An agent that invokes the refund API solely because the endpoint exists fails to reconcile the API specification with the governing policy.

4.3 API Execution and Response Handling

Once an API has been selected and invoked, the agent must correctly execute the call and process the returned result. While agent reasoning focuses on selecting appropriate APIs and constructing valid inputs, this stage concerns whether the API call is successfully carried out and whether its output is correctly handled. Failures at this stage may arise from execution issues such as authentication failures, rate limits, malformed requests, or server-side errors, as well as from incorrect interpretation of otherwise valid API responses. Prior work on tool-using and API-calling agents highlights the need to evaluate agents in interactive and stateful environments, where tool calls may fail, modify external state, or require recovery rather than simply producing a final answer [12, 14, 15, 53–55]. We therefore distinguish two aspects of this capability: API execution errors and API output errors.

4.3.1 API Execution Errors

Even when the correct API is selected and valid parameters are provided, execution may still fail if the API returns an error or non-success response. LLM agents must therefore detect such failures and respond appropriately, for example by retrying, switching strategies, or requesting additional information. This capability evaluates whether the agent can handle operational failures rather than assuming successful completion. Interactive API benchmarks such as ToolSandbox, AppWorld, and τ-bench are particularly relevant here because they evaluate agents under stateful workflows, execution failures, intermediate states, and policy-constrained interactions [15, 53, 54]. Typical cases include HTTP 404, 401, 429, or 500 responses, as well as timeouts and network failures. For example, if the task is "*Get the latest EUR to USD exchange rate*" and the call `get_rate(base="EUR", quote="USD")` returns {`"error": {"code": 429, "message": "Too Many Requests"`}}, the agent should recognize that the task has not been completed successfully. Instead of hallucinating a rate or failing silently, it should back off, retry within a bounded policy, or explain the limitation. Process-aware and failure-aware evaluation methods are useful for assessing such behavior because they inspect intermediate traces, retries, failure detection, and root-cause attribution rather than relying only on final task success [47, 79–81].

4.3.2 API Output Errors

A correct API call does not guarantee a correct final answer. The agent may invoke the appropriate API and receive a valid response, yet still produce an incorrect result due to misinterpretation of the output. This capability evaluates whether the agent can correctly extract relevant fields, preserve meanings such as units or formats, and avoid inferring information beyond what the API returns. Tool-use and API-execution metrics are therefore needed not only for tool selection and argument validity, but also for execution success and correct handling of API outputs [66, 82, 83]. For example, if the user asks "*What time is it in Tokyo right now?*" and the API call `get_time({city: "Tokyo"})` returns { `time: "19:30", timezone: "Asia/Tokyo"` }, the agent should respond consistently with the 24-hour time representation. Responding with "7:30 AM" would indicate that the agent misinterpreted the returned value

and produced an incorrect final answer despite the API call succeeding. Such errors motivate capability-specific evaluation of response interpretation, faithfulness to retrieved or returned information, and failure attribution across the execution trace [73, 84, 85].

5 Benchmark Construction

This section describes how the benchmark will be constructed to evaluate API-calling agent capabilities. It explains how tasks and test scenarios are generated and curated to simulate realistic API interactions and assess different aspects of agent reasoning, robustness, and reliability.

5.1 Overview of the Benchmark Construction Pipeline

The construction of API-calling benchmarks follows a pipeline that transforms source data into pairs of natural language tasks and executable API calls. These pairs represent user intents together with their corresponding structured API invocations, enabling the evaluation of an agent's ability to correctly map natural language requests to API specifications [55]. Table 1 illustrates examples of task–API call pairs from existing benchmarks. Once these API calls are executed, the returned information should enable the LLM agent to correctly respond to the corresponding user task.

Table 1 Examples of utterance–API call pairs from existing API-calling benchmarks.

User Task	**API Call**	**Dataset**

Get the latest closing prices and volumes for Apple Inc., Google LLC., and Microsoft Corporation in the New York Stock Exchange.	`get_stock_data(symbol="AAPL", data_points=["price","volume"])`
	`get_stock_data(symbol="GOOGL", data_points=["price","volume"])`
	`get_stock_data(symbol="MSFT", data_points=["price","volume"])`
		BFCL [86]

I am planning a family vacation in Italy. Please give me a list of Flixbus stations and search for available trips from Rome to Venice on 2022-04-30 for 6 people.	`GET Flixbus/stations`
	`GET Flixbus/search-trips`
	`?from_id=$rome_id`
	`&to_id=$venice_id`
	`&departure_date=2022-04-30`
	`&number_adult=6`
	`&currency=EUR`
		ToolBench [5]

I'm looking for a football player who scored 16 goals in the 2024–25 season, 1 goal in the UEFA Champions League 2024–25, 11 goals in the 2021–22 season, and 2 goals in the EFL Cup 2020–21. Who is this player?	`google_search.search(` `query="player scored 16 goals 2024-25 season with 1 goal UEFA Champions League 2024-25 and 11 goals 2021-22 season wih 2 goals EFL Cup 2020-21")`	MCP-Universe [87]

Methods for building benchmarks typically involve three stages: the selection of source data, the generation of task–API call pairs, and the application of quality-control mechanisms. The source data correspond to the initial seed resources used to define APIs, tasks, and execution scenarios, such as API documentation or existing datasets. The generation methods define the strategies used to produce natural language tasks together with their corresponding API calls from these sources. Finally, the quality-control stage ensures the correctness, consistency, diversity, and usability of the generated benchmark instances.

Both the generation methods and the quality-control mechanisms can be implemented using three main approaches: expert-based curation, crowdsourcing, and LLM-based curation. Expert-based approaches rely on the expertise of professionals such as developers, researchers, or computer science students to produce high-quality and semantically accurate data [11, 88]. However, these approaches are often time-consuming and difficult to scale.

Crowdsourcing approaches rely on contributions from non-experts, typically recruited through online platforms, to curate benchmark instances [48, 89]. This improves scalability and can increase linguistic diversity, particularly when contributors come from different linguistic backgrounds. However, crowdsourcing methods may introduce noise, including grammatical errors, inconsistencies, and incorrect API calls, increasing the importance of robust quality-control mechanisms [90].

LLM-based approaches use LLMs to automatically generate synthetic data, providing a cost-effective and scalable solution for benchmark construction [5, 6]. However, LLMs may introduce errors such as hallucinated API calls, incorrect parameter assignments, and violations of schema constraints (e.g., types, formats, or dependencies) [70]. In addition, the generated data may not fully reflect the complexity and distribution of real-world user tasks [91].

The remainder of this section details the types of sources, generation approaches, and quality control methods used in existing API-calling benchmarks.

5.2 Source Types

The source types refer to the different forms of seed data used to construct API-calling benchmarks. We found that the main source types include API documentation, existing benchmarks, API dependency graphs, and seed task–API call pairs.

API Documentation. API documentation describes the functionality, endpoints, parameters, and constraints of an API, enabling developers and agents to understand how to interact with it. It is commonly represented using OpenAPI Specifications (OAS), a standardized machine-readable format widely adopted by API developers [71]. API specifications are typically obtained from online hubs (e.g., RapidAPI, HuggingFace, APIs Guru, and SwaggerHub) or official developer websites.

Recently, the Model Context Protocol (MCP) has emerged as an alternative standard for exposing API schemas and tool interfaces to LLM agents at runtime. Benchmarks such as MCP-Bench [76] and MCP-AgentBench [77] use MCP server descriptions as their seed source, which are functionally analogous to OAS files. Figure 3 shows an example of API documentation for obtaining workout exercises.

Existing Benchmarks and Datasets. Existing datasets can also serve as source data for constructing new API-calling benchmarks, either by reusing resources from the API-calling domain or by adapting datasets from related domains. For example, within the API-calling domain, several works build upon ToolBench [5] by reusing its API documentation or generated

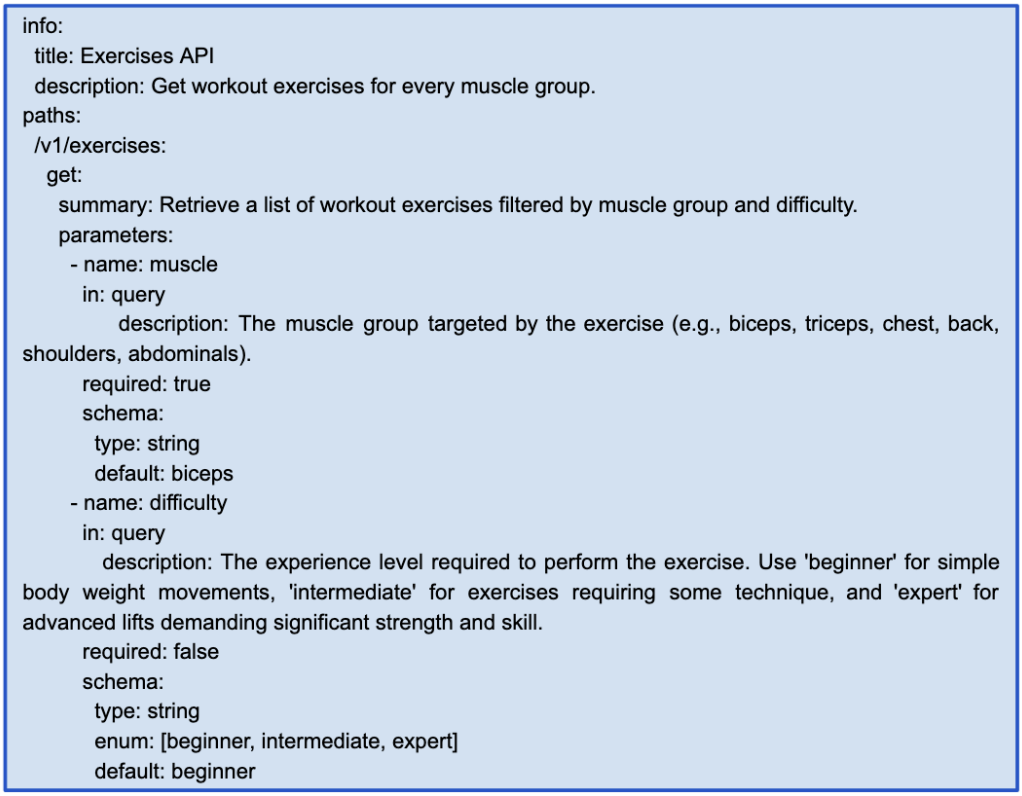

Fig. 3 Example of OpenAPI Specification of an API that retrieves Workout Exercises

task–API call pairs as seed resources for constructing new or extended benchmark instances [18, 92, 93].

Beyond API-calling datasets, benchmarks from related domains can also be adapted, including task-oriented dialogue datasets (e.g., MultiWOZ [94] and Schema-Guided Dialogue [95]), digital assistant interaction datasets, and text-to-SQL benchmarks. In such cases, their structured representations (e.g., intents, slots, workflows, or SQL queries) are transformed into corresponding API-call formats.

API Dependency Graphs. API dependency graphs are structured representations that explicitly encode dependencies between API endpoints, where nodes represent API operations and parameters, and edges capture relationships such as input/output parameter mappings between calls [72]. These graphs enable the representation of sequences of API calls required to achieve a goal, making them particularly useful as a seed source for generating composite tasks that require orchestrating multiple API calls.

For instance, Figure 4 illustrates a dependency graph in which three endpoints are chained together: `bookSeats` requires a `seat_id` parameter obtained from `getSeats`, which in turn requires a `flight_id` returned by `searchFlights` given an origin, destination, and date. Such dependency chains can serve as seed sources for generating tasks and corresponding API-call

sequences for requests such as *"Book an economy seat on the cheapest flight from Dublin to Abu Dhabi on March 3rd for John Smith"*.

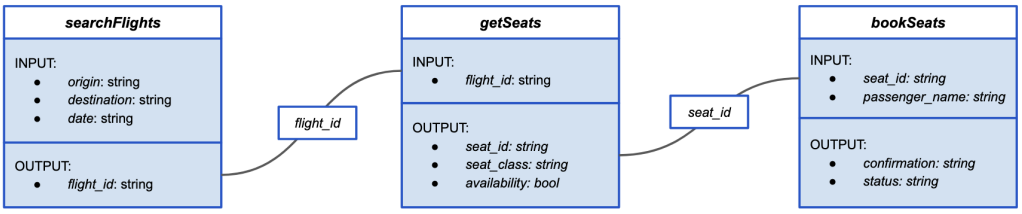

Fig. 4 Example of an API dependency graph where `searchFlights`, `getSeats`, and `bookSeats` are chained through output-to-input parameter mappings.

Seed Pairs. Seed pairs represent instances of user tasks together with their corresponding API calls used to fulfil those tasks. These pairs can be transformed, augmented, or paraphrased to construct benchmarks targeting different capabilities of LLM agents, such as handling ambiguous requests or linguistically diverse inputs. For example, NoisyToolBench [18] paraphrases seed tasks to introduce ambiguity, enabling the evaluation of how LLM agents perform when user intent is underspecified.

Real Human-Agent Conversation Data. Real human-agent conversation data refers to verbatim interactions recorded from production deployments of LLM-based agents, in which users issue natural language tasks and the agent executes corresponding API calls to fulfil them. For example, Jha et al. [96] proposes an automated curation pipeline that builds production-derived benchmarks from real developer-agent sessions in production.

5.3 Curation and Generation Methods

Generation methods represent the strategies used to produce user tasks and their corresponding API calls from the seed sources described in the previous subsection. We identified four main methods: generation from documentation, repurposing existing benchmarks, simulation, and paraphrasing.

Generation from Documentation. This method generates user tasks and their corresponding API calls using API documentation as seed sources, such as OAS files or MCP server descriptions. A curator (e.g., a crowd worker, an LLM, or a domain expert) is provided with API documentation and instructed to generate user tasks grounded in the API, together with corresponding API calls that satisfy the task requirements. Multiple API documentation files or API dependency graphs may also be provided to generate composite tasks requiring the orchestration of multiple API calls. This approach is widely adopted in existing benchmarks because it leverages standardized API representations commonly used in real-world systems [5, 97, 98].

Repurposing Existing Benchmarks and Datasets This method constructs API-calling benchmarks by transforming benchmarks originally developed for other domains, such as task-oriented dialogue, digital assistant interaction, or text-to-SQL generation. The transformation typically involves mapping the original task representations (e.g., database queries,

dialogue acts, or slot-value pairs) to corresponding API calls, while preserving the original natural language tasks or applying minor adaptations. For instance, a text-to-SQL dataset can be repurposed for API calling by treating SQL queries as structured analogues of API calls, where tables correspond to API endpoints and query predicates correspond to parameter values [99]. This approach reduces the cost of data collection and annotation by leveraging existing benchmark resources, although it may introduce domain mismatches or unrealistic API interactions that require additional quality control.

Simulation. This method generates tasks and API calls by simulating realistic user interactions, environments, workflows, or execution scenarios. Simulation may involve modelling different user goals, behavioural patterns, operational constraints, multi-step workflows, or dynamic environments. One common form of simulation relies on personas defined by sociodemographic attributes (e.g., native language, age, gender) and psychosocial attributes (e.g., occupation, hobbies, interests) [100]. This user-oriented conditioning promotes the generation of linguistically diverse and contextually varied tasks. For example, a budget-conscious traveller may write *find affordable flights with flexible dates from Abu Dhabi to Dublin*", while a premium traveller may request *book a direct business-class flight from New York to Paris*". This method is particularly relevant when LLMs are used as curators, as it enables models to simulate diverse user perspectives, workflows, and execution contexts [100, 101].

Paraphrasing. This method generates new tasks and API calls by rephrasing existing seed pairs. The transformation objective depends on the evaluation goal: a curator may paraphrase seed tasks to introduce linguistic diversity, malicious instructions, deliberate ambiguity, or other variations relevant to specific capabilities. For instance, ToolAlign [102] paraphrases seed tasks from ToolBench [5] to introduce harmful instructions involving privacy violations and unsafe behaviours, enabling the evaluation of LLM agents under different safety scenarios. This flexibility makes paraphrasing a versatile augmentation strategy capable of producing benchmarks that target a wide range of agent capabilities.

5.4 Quality Control

Quality-control mechanisms define the strategies used to ensure and improve the quality of benchmark instances [103–105]. We identified two main quality-control mechanisms: filtering and correction.

5.4.1 Filtering

This method removes instances that fail to satisfy predefined quality criteria [92, 106]. Multiple quality aspects may be considered during filtering, including the naturalness and fluency of user tasks, the correctness of API calls, and the consistency between user tasks and their corresponding API calls. Filtering can be performed using three main approaches: automatic validation, LLM-as-judge assessment, and human evaluation.

Automatic Validation. This approach applies rule-based, schema-driven, or checklist-based validation to discard invalid instances. Validation steps may include checking formatting inconsistencies (e.g., outputs not represented in JSON), argument parsing errors (e.g., invalid parameter or function names), violations of API schema constraints, response failures when invoking API calls, duplicate tasks, or inconsistencies between user tasks and API calls. For

instance, APIGen [92] filters instances that do not conform to JSON format and instances whose corresponding API calls return execution errors.

LLM-as-judge. This approach uses a separate LLM, or a collection of LLMs, to evaluate benchmark instances against predefined quality criteria and exclude those that fail to satisfy them. The LLM may be prompted to make a binary decision on whether an instance should be removed, or to score instances along one or more quality dimensions and discard those falling below predefined thresholds. The evaluation criteria may include aspects such as logical consistency, correctness of API calls, task solvability, completeness, naturalness, or adherence to constraints. For instance, Iskander et al. [106] used GPT-3.5 to remove task–API call pairs that fail to satisfy multiple quality attributes, such as whether all requests in the task are logically related and whether the task can be fulfilled using the ground-truth API calls.

Human Evaluation. Filtering can also be performed by crowd workers or domain experts who manually identify and discard instances that do not satisfy quality standards. For example, TaskBench [107] requested human experts to review instances and remove those that lacked naturalness, complexity, and semantic relevance to APIs. Typically, multiple annotators evaluate the same instances, and inter-annotator agreement metrics, such as Cohen's kappa, are computed to assess the consistency and reliability of the filtering decisions.

5.4.2 Correction

Rather than discarding low-quality instances, this method revises them to satisfy the required quality standards [108, 109]. It is particularly useful when errors are minor and can be corrected with limited modifications, such as unnatural task phrasing, incomplete constraints, inconsistent task–API mappings, incorrect parameter values, schema violations, or missing API arguments. This approach preserves potentially useful benchmark instances that would otherwise be removed during filtering, thereby improving dataset coverage and diversity.

Correction can be performed automatically by LLMs prompted to rewrite, repair, or complete problematic instances, or manually by human annotators who revise tasks and API calls to ensure correctness, consistency, naturalness, and adherence to API specifications and workflow constraints.

5.5 Proposed Design of a Capability-Based Pipeline for API-Calling Agent Benchmark Construction

This section presents a design of a pipeline for constructing API-calling benchmarks that evaluate multiple capabilities of API-calling agents. Figure 5 provides an overview of the proposed pipeline, which combines different source types, generation methods, and quality-control mechanisms.

Task–API call pairs are first generated from three source types. API documentation is used to build API dependency graphs, which provide structured representations of dependencies between API endpoints. Both the dependency graphs and the original API documentation are then provided to LLMs to generate task–API call pairs, enabling the scalable construction of simple and composite tasks. The same API documentation can also be provided to crowd workers, who generate more diverse tasks that reflect how users with different backgrounds formulate requests. Additional sources include task–API call pairs derived from existing API-calling benchmarks and cross-domain benchmarks repurposed for the API-calling domain.

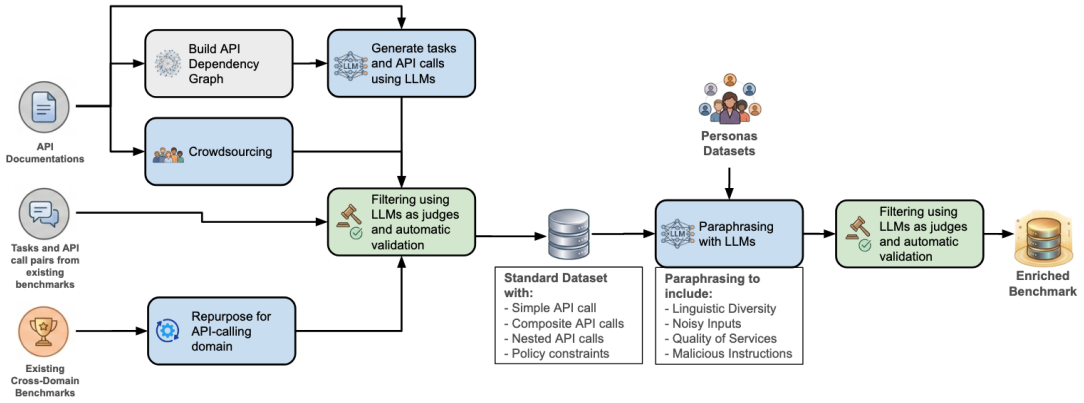

Fig. 5 Diagram of proposed pipeline for building API-calling agent benchmarks.

The generated instances are then filtered using LLM-as-judge approaches and automatic validation mechanisms. LLM-as-judge approaches evaluate quality attributes of task–API call pairs, such as naturalness, fluency, consistency, and correctness, while automatic validation ensures that generated API calls conform to the expected JSON format and do not contain hallucinated parameters or schema violations. This step produces a core dataset comprising simple tasks, composite tasks, and tasks requiring nested API calls.

The core dataset is subsequently enriched through LLM-based paraphrasing and persona-conditioned augmentation techniques, introducing linguistic diversity, noisy and malicious inputs, and quality-of-service constraints. This enrichment extends the benchmark coverage to capabilities of LLM agents that may not be sufficiently represented in the initial dataset. A second round of LLM-as-judge assessment and automatic validation is then applied to the enriched instances to ensure that quality standards are maintained in the final benchmark.

6 Agent Assessment Framework

This section presents the capability-based framework used to evaluate API-calling agents. Rather than measuring only whether a final task is completed, the framework evaluates whether the agent demonstrates the intended capabilities, follows a valid execution process, and behaves reliably under realistic operating conditions. This distinction is important because an agent may appear successful while still selecting the wrong API, constructing invalid parameters, skipping a required clarification step, or violating policy constraints. Accordingly, the evaluation is organized around four complementary dimensions: outcome, trace, capability, and environment, while treating reliability as a cross-cutting property measured across repeated runs. Figure 6 provides an overview of the proposed capability-based assessment framework [73].

6.1 Assessment Dimensions

The framework evaluates API-calling agents across four complementary dimensions (see Figure 7): outcome, trace, capability, and environment. Together, these dimensions provide a more comprehensive assessment of agent behaviour than outcome-based evaluation alone.

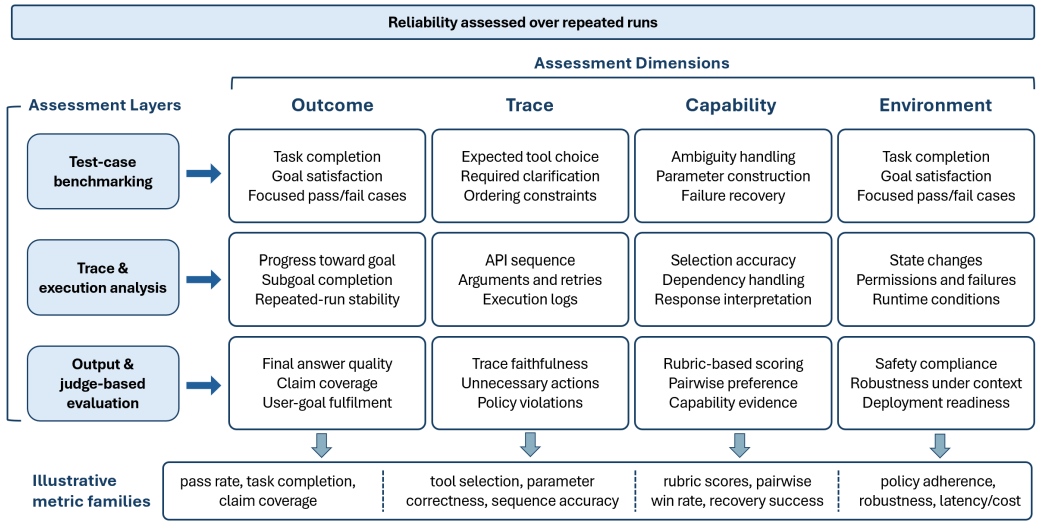

Fig. 6 Capability-based Agent Assessment Framework.

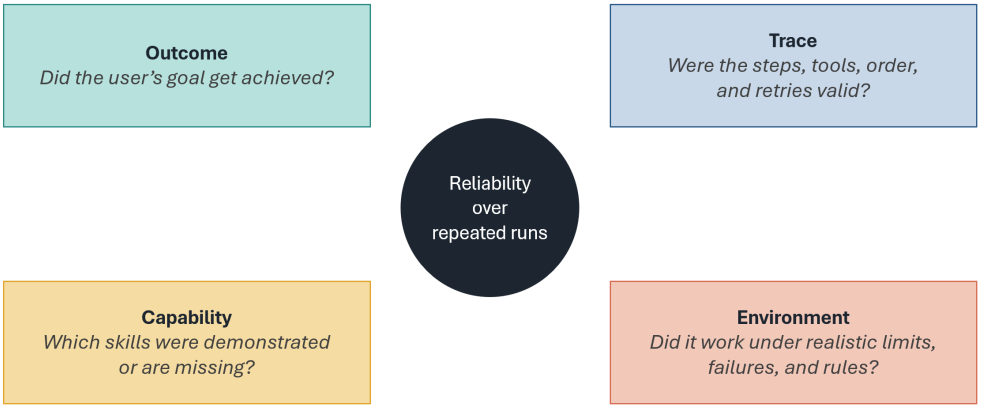

Fig. 7 Dimensions of the Assessment Framework.

The first dimension is **outcome**, which measures whether the user's goal was achieved. Outcome assessment captures task completion, goal satisfaction, and answer quality. It remains necessary because the final deliverable is the most direct signal of utility. However, outcome metrics alone are insufficient for API-calling agents, since different traces may lead to the same visible result, and some apparently successful outcomes may still conceal policy violations, invalid intermediate reasoning, or accidental success.

The second dimension is **trace**, which evaluates the sequence of internal and external actions taken by the agent during execution. For API-calling agents, the trace includes clarification turns, selected APIs, constructed arguments, retries, recovery steps, intermediate tool outputs, and the ordering of calls in multi-step tasks. Trace-level assessment is essential because many capability failures are only observable in execution logs. For example, an agent may eventually

answer correctly while still invoking an unnecessary tool, violating a dependency constraint, or issuing redundant calls. This process-oriented perspective is increasingly emphasized in recent work on agent evaluation [14].

The third dimension is **capability**, which evaluates whether the execution demonstrates the intended competences defined in Section 3. In our setting, this includes understanding ambiguous or incomplete inputs, reasoning over QoS constraints, selecting the correct API, constructing valid parameters, respecting dependencies and ordering constraints, handling execution failures, and correctly interpreting API responses. For multi-turn dialogue tasks, this dimension should also assess whether the agent maintains an accurate representation of the current user goal. Recent work on intent rewriting shows that explicit intermediate intent representations can improve downstream planning by making the user's latest goals more concise, unambiguous, and up to date. For this reason, capability assessment in dialogue-based agents should not be restricted to final plans alone, but should also evaluate the quality of intermediate intent representations when such modules are used [84].

The fourth dimension is **environment**, which evaluates whether the agent behaves appropriately under realistic operating conditions. This includes sandboxed or mocked API environments, dynamic multi-turn settings, policy-constrained enterprise scenarios, and adversarial inputs such as malicious prompt injections. Environment-level assessment is needed because the same agent may behave differently depending on available tools, permissions, latency, failures, and security constraints. In practice, evaluation often progresses from controlled environments to more realistic settings as trust in the system increases. Enterprise-oriented surveys similarly emphasize role-based access, compliance, and long-horizon interaction as central evaluation concerns [14].

Across all four dimensions, reliability is assessed as a repeated-run property rather than a single score. Since LLM-based agents are stochastic, a one-off successful execution does not necessarily indicate dependable behaviour. Reliability should therefore be measured by repeating the same task, or semantically equivalent variants of the same task, and analysing the consistency of outcomes, traces, and policy adherence. This is particularly important in deployment settings where occasional success is insufficient if the agent cannot produce stable and auditable behaviour over time.

6.2 Assessment Methodology

This section describes how the proposed framework operationalizes the assessment dimensions introduced above. The methodology combines three complementary forms of evaluation (see Figure 8). First, test-case and benchmark-based assessment evaluates whether the agent demonstrates the target capabilities across controlled cases and realistic benchmark tasks. Second, trace-based and execution-based assessment examines the process followed by the agent, including selected APIs, constructed parameters, intermediate states, execution errors, and ordering constraints. Third, output-, preference-, and verification-based assessment evaluates the final response, open-ended outputs, human or LLM-judge preferences, and verifier behaviour when such components are used. The goal is to identify which capabilities explain a success or failure.

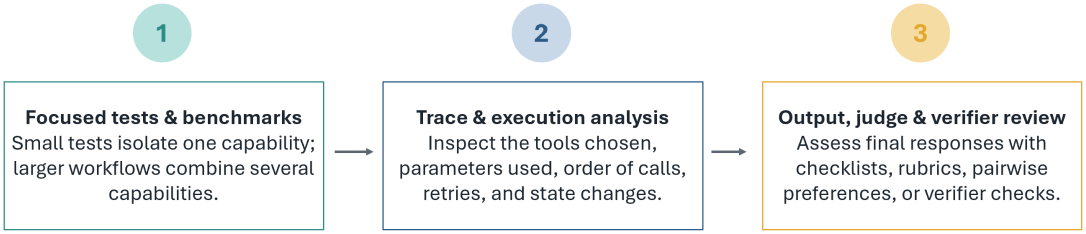

Fig. 8 The assessment framework operationlizes assessment dimensions into methodologies.

6.2.1 Test-case and benchmark-based assessment

The framework combines focused test cases and multi-step workflow benchmarks to evaluate different aspects of API-calling agent behaviour. Focused test cases act similarly to unit tests for agents by isolating one capability in one situation, specifying the expected behaviour, and checking whether the agent satisfies it. For example, an ambiguity-handling case may present the request "*Send a message to John saying I arrived*" and the expected behaviour is that the agent asks which John is intended before acting. Individual test cases are grouped into evaluation sets targeting broader capabilities or risk areas, such as incomplete inputs, conflicting constraints, missing parameters, policy-restricted actions, or tool failures. This design supports diagnostic analysis because failures can be localized to specific capabilities instead of being hidden inside aggregate success rates.

Beyond focused cases, the framework also includes multi-step workflow benchmarks to measure how agents perform on realistic tasks that require multiple capabilities simultaneously. These include cross-domain tasks, multi-step workflows, and multi-turn interactions. Such tasks are particularly useful for evaluating orchestration, dependency satisfaction, and recovery under execution uncertainty. The benchmark may also include dedicated modules for related agentic capabilities that interact with API calling, such as planning, retrieval-augmented generation, and instruction following, when these are part of the target system. TelAgentBench provides a representative example of such a modular design, where reasoning, planning, action, retrieval, and instruction following are evaluated separately within a unified benchmark [110]. For example, a travel-planning task may require the agent to search for flights, compare hotel options, check weather conditions, and then produce a final itinerary. Such a task tests more than API selection: it also tests whether the agent decomposes the request into subgoals, calls the APIs in a sensible order, carries information from one step to the next, and produces a final answer that reflects all requested constraints.

For input robustness, the framework also includes paired or contrastive cases. These cases vary the phrasing of the same request through paraphrases, typos, informal language, contradictory requirements, or attribute changes while preserving the intended evaluation target. The objective is to verify that equivalent requaests lead to equivalent behaviour, and that noisy or underspecified requests trigger clarification rather than premature execution. In multi-turn settings, evaluation should also include intent drift and multi-intent conversations, since these are common sources of planning failure in practice. RECAP further shows that such settings require explicit assessment of whether the system continues to reflect the user's latest goal rather than outdated or irrelevant turns from the dialogue history [84].

The framework can also support dynamic test generation, where new test cases are created or selected adaptively based on previously observed failures. Instead of relying only on a fixed benchmark, the evaluator can begin with seed tasks and then mutate them to stress specific capabilities. For example, if an agent repeatedly fails to handle ambiguous contacts, the benchmark can generate additional variants such as "Send the file to Alex", "Message John that I am late", or "Book the usual hotel", where multiple valid interpretations exist and clarification is required. Similarly, if the agent fails on dependency constraints, the evaluator can generate more tasks that require authentication, lookup, or state initialization before execution. This idea is inspired by capability-oriented testing and robustness evaluation, where test generation is guided by failure evidence, coverage gaps, or slices of difficult examples [111, 112]. In the proposed framework, dynamic test generation is useful for two purposes: improving benchmark coverage over under-tested capabilities and producing targeted regression tests for failures discovered during agent evaluation.

6.2.2 Trace-based and execution-based assessment

The framework also evaluates execution traces produced during agent interaction and tool use. Static trace analysis compares the agent's selected tools, argument structures, and call ordering against reference traces or expected constraints. This is appropriate for capabilities such as API selection, dependency adherence, ordering constraints, and required clarification behaviour. Trace-based analysis can also be expressed structurally using graph or program representations, enabling comparison of predicted workflows against reference workflows rather than only comparing final answers. This is particularly useful when evaluation targets orchestration quality rather than endpoint success alone [85]. For example, if the API returns three hotel options with different prices, distances, and ratings, a checklist can verify whether the final response includes all required fields, while a rubric can judge whether the recommendation properly balances the user's stated preference, such as minimizing cost or staying close to the airport. In this case, exact-match evaluation would be too rigid because several responses may be acceptable, but the evaluator still needs a structured way to judge quality.

However, static trace checks are not sufficient on their own. In many cases, tool calls must be executed in a sandboxed environment to determine whether they are semantically correct. Syntax-level checks may fail to detect incorrect enumerated values, hallucinated arguments, or semantically invalid calls. Execution-based evaluation therefore runs the predicted API calls and evaluates the resulting behaviour, providing a more grounded assessment of tool use. This is especially important for API-calling agents because correctness often depends on real outputs, state transitions, and side effects rather than only on syntactic conformity. Recent assessment frameworks similarly combine static analysis with dynamic execution to capture failures that outcome-only evaluation overlooks [73].

For dynamic multi-step tasks, the framework tracks progress throughout execution rather than only evaluating the final outcome. Useful signals include matched calls, missing calls, completion status, recovery after failures, and whether the agent terminates appropriately. More generally, dynamic protocols can evaluate whether the agent selects the correct next action given the current execution state, respects stateful workflow constraints, and recovers from failures without deviating from policies or user objectives.

6.2.3 Output-, preference-, and verification-based assessment

The proposed framework in Figure 6 also evaluates outputs using checklists, acceptance criteria, and rubrics. Checklists are appropriate when the expected behaviour can be decomposed into required elements, such as preserving requested content, asking a clarification question, or satisfying a user constraint. Rubrics are more suitable when outputs are partially open-ended, such as recommendations or generated responses that must balance correctness, completeness, and relevance. Judge-based review, whether performed by humans or LLM evaluators, is useful for assessing reasoning quality, safety, or other aspects that are difficult to express through exact-match tests. For example, if the API returns three hotel options with different prices, distances, and ratings, a checklist can verify whether the final response includes all required fields, while a rubric can judge whether the recommendation properly balances the user's stated preference, such as minimizing cost or staying close to the airport. In this case, exact-match evaluation would be too rigid because several responses may be acceptable, but the evaluator still needs a structured way to judge quality.

For dialogue planning tasks, pairwise preference evaluation provides an additional and often stronger assessment signal. RECAP evaluates alternative rewrites and downstream plans by comparing two candidate plans produced from the same dialogue and determining whether one plan better reflects the latest intent, avoids fabrication, provides sufficient granularity, remains complete, and preserves logical order. This form of comparative assessment is particularly useful when multiple outputs may be acceptable but differ in planning utility. It also enables scalable LLM-as-a-judge evaluation once a human-annotated preference set is available [84].

Finally, when the target system includes a self-checking or verifier component, the framework should evaluate verification as a distinct capability. A verifier should be assessed according to its ability to accept correct candidates and reject incorrect ones, rather than only according to downstream task success. This is relevant for API-calling agents that rerank plans, validate intermediate outputs, or filter uncertain actions before execution. Verification-based assessment therefore complements direct task evaluation, particularly in high-stakes or long-horizon workflows [113]. For example, an agent may generate two possible plans before execution: one that directly books a flight and another that first asks the user to confirm the travel date because the request is underspecified. A verifier should prefer the second plan when the missing information is required for safe execution. Evaluating the verifier separately makes it possible to determine whether failures arise from the original planner, the verifier, or the interaction between them.

6.3 Evaluation Metrics

The proposed framework uses six families of metrics (see Figure 9) to evaluate API-calling agents from complementary perspectives. Since agent performance cannot be fully captured by final task success alone, the metrics should assess both the outcome of the task and the process by which the agent reaches that outcome. For example, an agent may return a correct final answer while using an unnecessary API, violating a dependency constraint, or ignoring a required clarification step. Conversely, an agent may follow a valid trace but still misinterpret the API response in the final answer. The framework therefore combines outcome,

tool-use, trace, capability, robustness, reliability, efficiency, safety, and preference-based metrics. Table B2 summarizes the main assessment families, their evaluation focus, representative metrics, and the studies that motivate each family.

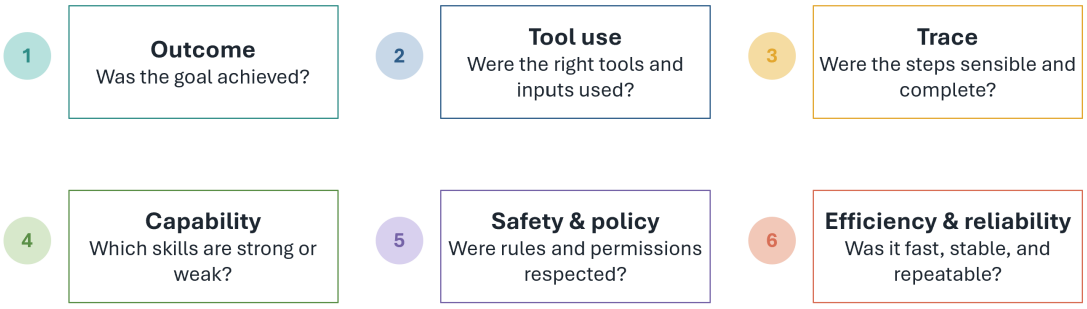

Fig. 9 The proposed assessment framework uses six metric families.

As shown in Table B2, the most direct form of evaluation is **outcome-based task success**. Outcome metrics capture end-task quality and include pass rate, task completion, task-goal completion, execution accuracy, exact-match variants, and claim coverage. These metrics answer the basic question of whether the user goal was achieved. For example, in a flight-booking task, an outcome metric checks whether the final booking satisfies the requested destination, date, and passenger constraints. However, outcome metrics should not be interpreted in isolation, since they may hide invalid intermediate behaviour or accidental success.

Tool-use and API execution metrics evaluate the correctness of the actions taken by the agent. These include invocation accuracy, tool-selection accuracy, tool-discovery accuracy, parameter correctness, parameter-name F1, parameter-value accuracy, API-call validity, execution success, and dependency adherence. For example, if the user asks for the weather in Paris, the agent should select the weather API rather than a time-zone API, and it should construct the required parameter using a valid city value. For large tool repositories, retrieval-style metrics such as MRR and NDCG can also be used to measure whether the correct tool appears near the top of the agent's candidate list. These metrics are particularly important because API-calling failures often arise before the final response is produced.

Trace and process metrics assess the sequence of reasoning and actions followed by the agent. Rather than only checking whether the final answer is correct, these metrics examine intermediate steps such as selected APIs, constructed arguments, retries, clarification turns, and state transitions. For structured plans or workflows, graph-based metrics such as Node F1, Edge F1, graph edit distance, and normalized edit distance can be used to compare predicted and reference workflows. In dynamic environments, progress rate, step success rate, failure-detection recall, and root-cause accuracy provide a finer-grained view of how effectively the agent advances toward the goal. For example, in a shopping workflow, a trace metric can detect whether the agent called checkout before adding an item to the cart, even if the final response claims that the purchase was completed.

Capability-oriented metrics connect evaluation results back to the capability space defined in Section 3. Instead of reporting only aggregate performance, the framework should compute capability-specific scores for input understanding, API selection, parameter construction, dependency handling, policy compliance, recovery, and response interpretation. These metrics can be implemented through capability-specific test cases, capability oracles, or benchmark slices. For example, ambiguity-handling accuracy measures whether the agent asks for clarification when a request such as "Send a message to John" has multiple valid interpretations. Capability coverage can also be reported to indicate which capabilities are represented in the benchmark and which remain under-tested.

Robustness and dynamic testing metrics evaluate whether performance remains stable under realistic input and environment variation. These metrics include robustness-slice performance, perturbation degradation, paraphrase stability, performance under ambiguity or unanswerability, and adversarial robustness. For example, an agent should behave consistently when the same request is expressed as a command, a question, or an informal message containing typos. Dynamic testing can further use observed failures to generate additional targeted cases, such as more ambiguity-heavy tasks when the agent repeatedly acts prematurely without clarification. This makes robustness evaluation not only a reporting mechanism, but also a way to improve benchmark coverage over time.

Failure attribution and diagnosis metrics identify where failures occur and which capability, step, or component is most responsible. These include root-step accuracy, root-capability accuracy, agent-level attribution accuracy, failure cluster purity, feedback scores, and diagnostic agreement with human annotators. Such metrics are useful when an agent fails in a long workflow where errors propagate across steps. For example, if an agent books the wrong hotel because it misread the user preference before calling the booking API, the failure should be attributed to input understanding or preference handling rather than to API execution. This type of diagnostic signal is especially useful for leaderboard settings because it distinguishes models that fail for different reasons even when their overall pass rates are similar.

Preference and quality metrics are used when outputs are open-ended or when multiple plans may be acceptable. These include rubric scores, pairwise win rate, tie rate, LLM-as-judge scores, semantic similarity, graph edit distance, and ranking-based scores such as Bradley–Terry estimates. For planning tasks, pairwise preference judgments can capture aspects that are not well represented by exact-match scoring, such as whether a plan reflects the latest user intent, avoids fabricated steps, provides sufficient detail, and preserves logical order. For artifact-rich or open-ended tasks, these metrics allow the framework to compare output quality without requiring a single rigid reference answer [84, 85].

Retrieval and grounding metrics are relevant when the agent relies on external documents, memory, API documentation, or retrieved knowledge to support its decisions or final responses. In such cases, standard retrieval metrics such as precision, recall, and F1 remain useful, but response-level measures such as answer correctness, factual correctness, response relevance, and faithfulness should also be included. These metrics are particularly important when API-calling is combined with retrieval-augmented or knowledge-grounded assistance. For example, if an agent retrieves API documentation before constructing a call, grounding metrics can check whether the selected endpoint and parameter values are supported by the retrieved documentation.

Safety, policy, and trustworthiness metrics evaluate whether the agent remains compliant under operational constraints. Relevant measures include policy adherence, safety violations, guardrail violations, authorization failures, blocked-action correctness, introduced vulnerabilities, benchmark validity, contamination controls, and confidence intervals. In enterprise settings, these metrics should explicitly reflect approval workflows, role-based permissions, and domain-specific compliance requirements, since an agent may otherwise appear successful while still violating rules that would make it unsuitable for deployment. For example, an agent that grants administrator access without checking approval may complete the requested action, but it should receive a policy violation rather than a successful score [14, 73].

Efficiency and reliability metrics assess whether the agent is practical and dependable. Efficiency measures include time efficiency, token efficiency, duplicate calls, Time To First Token, end-to-end latency, monetary cost, and redundant tool use. Reliability measures include repeated-run consistency, robustness to perturbations, stability across multi-turn interactions, and trajectory reliability. When a verifier is present, additional metrics such as true positive rate, true negative rate, balanced accuracy, and verification gain can quantify whether the verifier improves the quality of retained outputs rather than merely filtering aggressively. These metrics are important for deployment and leaderboard design because two agents may achieve the same pass rate while differing substantially in cost, latency, stability, and safety [113].

6.4 Mapping Assessment Methods to Agent Capabilities

The assessment methods above can be mapped to the capabilities introduced in Section 3. Input-related capabilities, such as task complexity, domain diversity, linguistic diversity, and noise handling, are best evaluated through focused test cases, paired perturbation tests, and multi-turn dialogue scenarios. In settings with evolving dialogue, intent-state quality should also be assessed, for example by verifying whether a rewritten or summarized intent remains aligned with the user's latest goals.

Capabilities related to agent reasoning, including API selection, parameter construction, dependency handling, and ordering constraints, are best evaluated through trace-based validation combined with execution-based checks. These capabilities are inherently process-oriented: an agent must not only reach the correct result, but must do so through valid and policy-compliant actions. Accordingly, outcome metrics should be complemented with selection, parameter, sequence, and progress metrics.

Capabilities related to execution and response handling, such as failure recovery and API output interpretation, require dynamic execution protocols in sandboxed environments. These protocols should expose the agent to realistic API errors, missing parameters, and stateful workflows, and should evaluate whether the agent retries appropriately, requests clarification when needed, and avoids hallucinating successful execution.

Finally, capabilities related to malicious instructions, safety, and policy compliance require environment-aware evaluation with explicit guardrails and organizational constraints. More broadly, if the target agent includes additional components such as retrieval, instruction following, memory, or verifier modules, the same framework can be extended with dedicated assessment methods and metrics for those components while preserving the same core principle: evaluation should remain capability-based, diagnostic, and grounded in realistic execution conditions rather than relying solely on final task success.

6.5 Proposed Design for Capability-Based Assessment of API-Calling Agents

Figure 10 illustrates the proposed design of a pipeline for capability-based assessment of API-calling agents. The pipeline operationalizes the assessment dimensions introduced in this section by connecting capability specification, benchmark construction, dynamic test generation, execution monitoring, and leaderboard reporting into a single evaluation process. The objective is to ensure that API-calling agents are evaluated not only by whether they complete tasks, but also by whether they demonstrate the underlying capabilities required for reliable operation, such as intent understanding, API selection, parameter grounding, execution recovery, and policy compliance.

Fig. 10 Capability-based agent assessment pipeline.

To make this assessment process operational, Figure 11 expands the high-level pipeline into nine concrete steps. These steps describe how the framework moves from defining the capabilities to be assessed, to constructing and enriching benchmark tasks, validating generated tests, executing agents in controlled environments, collecting execution traces, and finally producing diagnostic scores and leaderboard reports. This stepwise view clarifies the role of each stage in the assessment process and shows how failures observed during evaluation can be fed back into the pipeline to generate more targeted tests. The following subsections describe each step in detail.

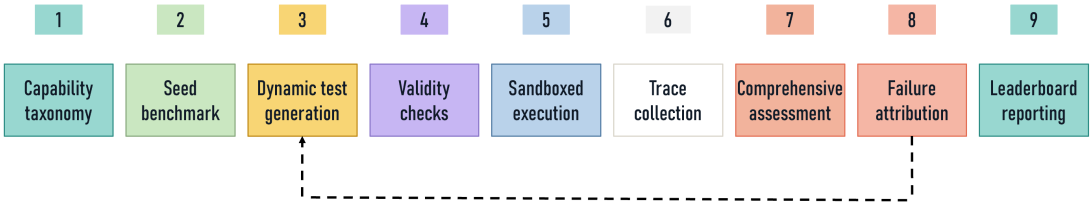

Fig. 11 Nine-step pipeline for capability-based assessment of API-calling agents.

Capability Taxonomy. The pipeline begins with the definition of a capability taxonomy. This taxonomy is derived from the capabilities introduced in Section 4 and provides the organizing structure for the assessment. Each benchmark instance is associated with one or more capability labels, allowing evaluation results to be aggregated at the capability level rather than

only at the task level. This design enables diagnostic analysis, since failures can be traced back to specific capability gaps such as incorrect API selection, invalid parameter construction, missing clarification, or failure to recover from an API error. Such capability-level organization is consistent with recent work emphasizing that agent evaluation should move beyond binary task completion and assess the behaviour of agentic systems across multiple components, including language models, tools, memory, and environment interactions [14, 73].

Seed Benchmark Construction. The second stage constructs a set of seed benchmarks. These seeds correspond to the static benchmark constructed in Section 5. They are created from API documentation, MCP server descriptions, API dependency graphs, existing benchmarks, and manually curated task-API call pairs. The seed benchmark is designed to cover both focused cases, which isolate individual capabilities, and composite cases, which require the agent to combine multiple capabilities in realistic workflows. For example, a focused case may test whether the agent asks for clarification when a required parameter is missing, while a composite case may require the agent to select multiple APIs, satisfy dependency constraints, execute them in the correct order, and synthesize a final response. This combination of focused and composite cases allows the benchmark to support both fine-grained diagnosis and realistic end-to-end evaluation.

The seed benchmark also provides the reference structure used by later stages of the pipeline. Each seed instance contains the user task, capability labels, available APIs, expected API calls or trace constraints, expected state changes, and scoring rules. This structure enables the benchmark to serve as a validated base set from which dynamic test cases can be generated. In this way, the static benchmark is not replaced by dynamic assessment; rather, it becomes the foundation for controlled transformations, perturbations, fuzzing-based mutations, and adaptive test generation.

Dynamic Test Generation. The third stage performs dynamic test generation. Rather than relying only on a fixed benchmark, the pipeline generates additional test cases from the seed instances using transformation, perturbation, adversarial mutation, environment mutation, and fuzzing-inspired techniques (see Figure 12). The goal is to leverage the static benchmark as a source of executable and validated seed cases, then systematically expand it into a broader test space that exposes failures under realistic variation.

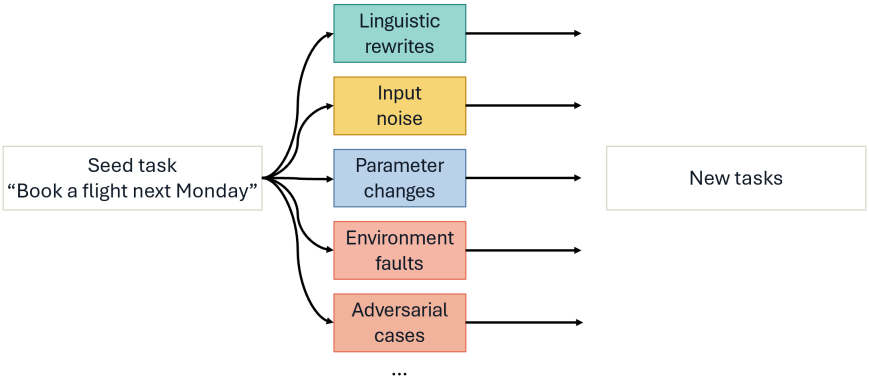

Fig. 12 Assessment pipeline step 3: dynamic test generation.

Linguistic transformations generate alternative surface forms of the same task while preserving the intended user goal. These include lexical substitutions, syntactic rewrites, changes in voice or sentence structure, persona-conditioned rewrites, style changes, and paraphrases. For example, a request such as "Book a flight from Dublin to Abu Dhabi next Monday" may be rewritten as "I need to travel from Dublin to Abu Dhabi next Monday; can you arrange the flight?". Such transformations test whether the agent is robust to linguistic diversity rather than memorizing benchmark phrasing.

Input perturbations introduce noise or uncertainty into otherwise valid requests. These include typos, spelling errors, grammatical mistakes, reordered constraints, irrelevant context, incomplete values, ambiguous references, contradictory requirements, and informal or emotional wording. These perturbations target input-understanding capabilities such as ambiguity handling, incompleteness detection, clarification, and robustness to noisy user requests. In some cases, the expected behaviour remains the same as the seed task; in others, the perturbation intentionally changes the expected behaviour, for example by requiring the agent to ask for clarification before executing an API call.

Schema- and parameter-level mutations modify API-relevant attributes while preserving the overall task structure. These mutations vary required parameters, optional parameters, argument formats, enumerated values, nested objects, and combinations of API fields. For example, a travel task can be mutated by changing the date, destination, cabin class, number of passengers, or adding a constraint such as lowest price or shortest duration. This mechanism supports systematic coverage of API schemas, parameter combinations, and edge cases that may be underrepresented in manually written tasks.

Adversarial mutations generate test cases that challenge the agent's ability to resist unsafe or misleading instructions. These include prompt injections in user requests, malicious content returned by tools, misleading API descriptions, distractor tools with similar names, conflicting instructions, and attempts to bypass policy constraints. To generate such cases systematically, the benchmark can use a fuzzing-inspired process: starting from an unsafe or misleading instruction, it mutates the wording while preserving the underlying adversarial intent. The mutation process may replace words with synonyms, change sentence structure, insert irrelevant context, or rephrase the instruction in a different style. This produces diverse adversarial variants that express the same harmful or misleading goal in different ways, enabling continued red-teaming of API-calling agents as models improve.

Environment mutations alter the execution conditions under which the agent operates. These include API timeouts, null responses, authentication failures, rate limits, stale data, permission errors, malformed outputs, unavailable tools, changed schemas, and unexpected state transitions. Such cases evaluate whether the agent can recover from execution uncertainty rather than assuming that all APIs behave correctly. They also test whether the agent can distinguish between successful task completion and partial or failed execution.

Overall, dynamic test generation can be implemented as an iterative fuzzing loop. Starting from a seed task, the pipeline applies one or more mutation operators to produce candidate test cases. These candidates are then validated, executed against agents, and analyzed for failures. Mutants that reveal new failures or increase capability coverage are retained as valuable test cases, while invalid or redundant mutants are discarded. Retained mutants can be further mutated in later rounds, allowing the benchmark to evolve beyond a static test set and continuously search for capability weaknesses, robustness gaps, and safety vulnerabilities.

Benchmark Validity Checks. The fourth stage validates the quality of both seed and dynamically generated tasks (see Figure 13). This stage is necessary because automatic generation, perturbation, and fuzzing may introduce invalid tasks, ambiguous ground truth, or shortcuts that allow agents to pass without demonstrating the target capability. We therefore include explicit benchmark validity checks before tasks are used for scoring. Task validity requires that a task be solvable if and only if the agent possesses the intended capability. Outcome validity requires that the evaluation result correctly reflect task success. These checks include verifying tool accessibility, resetting environment state between runs, isolating agents from ground-truth information, confirming task solvability through an oracle or reference execution process, and ensuring that answer matching, state matching, tests, or judge-based scoring cannot be exploited by trivial behaviours. This follows the Agentic Benchmark Checklist, which identifies task validity, outcome validity, and reporting validity as central conditions for rigorous agentic evaluation [75].

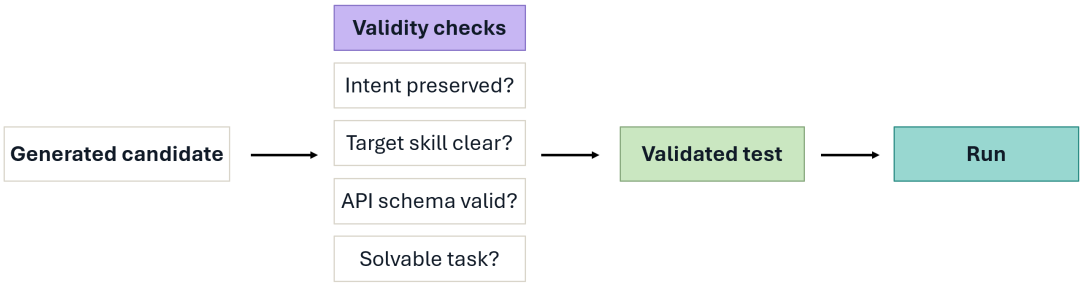

Fig. 13 Assessment pipeline step 4: validity checks.

For dynamically generated tests, additional validity checks are required. First, an *intent-preservation check* determines whether a transformed or paraphrased task still expresses the same underlying goal as the seed task. Second, a *capability-target check* verifies whether the mutation preserves the original capability target or intentionally shifts the task to a new target, such as from direct execution to clarification. Third, a *schema-consistency check* ensures that mutated API calls remain compatible with the API specification unless the purpose of the test is to evaluate missing or invalid parameters. Fourth, an *oracle-update check* updates the expected state, reference answer, or trace constraint when a mutation changes task attributes such as date, location, quantity, or policy constraints. These checks ensure that dynamic generation increases evaluation coverage without weakening the validity of the benchmark.

Sandboxed Execution. The fifth stage executes agents in a sandboxed environment (see Figure 14). Each agent is given access to the tools and APIs required by the task, together with plausible distractor tools when appropriate, enabling evaluation of tool selection under realistic conditions. The sandbox records the initial environment state, executes API calls, simulates faults, and captures the final state after the run. To account for stochastic behaviour, each task may be evaluated across repeated trials and semantically equivalent variants. This allows the framework to measure not only one-off success, but also consistency, robustness, and reliability. Sandboxed execution is particularly important for API-calling agents because correctness often

depends on actual tool responses, state transitions, authorization constraints, and side effects rather than only on syntactic validity of generated calls.

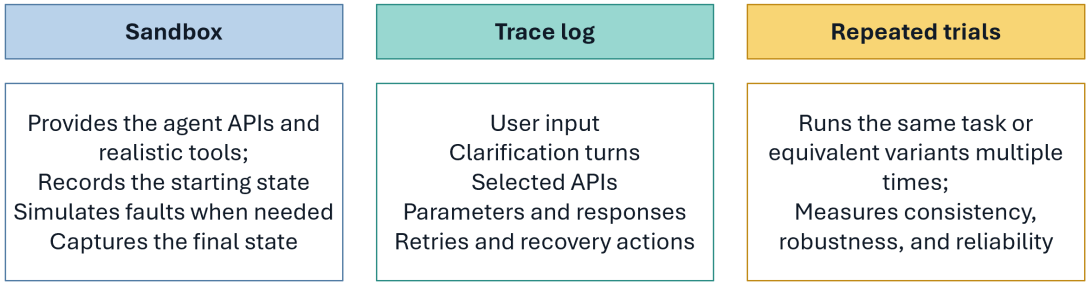

Fig. 14 Assessment pipeline steps 5 and 6: sandboxed execution and trace collection.

The sandbox also supports controlled fault injection. During execution, the framework can selectively trigger API errors, delayed responses, invalid return values, permission failures, or unavailable services. These faults can be generated according to predefined test scenarios or adaptively based on the agent's previous behaviour. This enables execution-level stress testing of recovery capabilities, such as retrying, backtracking, choosing alternative tools, asking the user for clarification, or safely terminating when the task cannot be completed.

Trace Collection. The sixth stage collects execution traces. A trace includes the user input, clarification turns, intermediate plans, selected APIs, constructed arguments, API responses, retries, memory accesses, recovery actions, final answer, and final environment state. Trace collection enables process-level assessment, making it possible to determine whether the agent reached the correct answer through valid and reliable behaviour. This is important because outcome-only evaluation can hide intermediate errors, accidental success, redundant tool use, policy violations, or invalid recovery strategies. Recent work on step-level and graph-based agent evaluation similarly argues that agent executions should be evaluated as structured workflows rather than as isolated final outputs [80].

Comprehensive Assessment. The seventh stage performs comprehensive assessment across the proposed evaluation dimensions (see Figure 15). Outcome assessment measures whether the final user goal was achieved. Trace assessment evaluates the correctness of the plan, tool choices, parameter values, call ordering, and recovery actions. Capability assessment aggregates results over the capability labels associated with each task. Robustness assessment compares performance across original, paraphrased, perturbed, adversarial, and fault-injected variants. Reliability assessment measures consistency across repeated runs. Together, these assessment dimensions provide a more complete view of agent behaviour than task success alone. When a verifier or self-checking component is present, the framework can also assess the verifier separately using acceptance and rejection metrics, since verification quality may differ from generation quality and can affect downstream reliability [113].

The dynamic nature of the benchmark enables additional assessment signals. For example, *paraphrase stability* measures whether semantically equivalent inputs produce equivalent behaviour. *Perturbation robustness* measures the performance drop under noisy, ambiguous, or incomplete inputs. *Adversarial robustness* measures whether the agent remains safe and

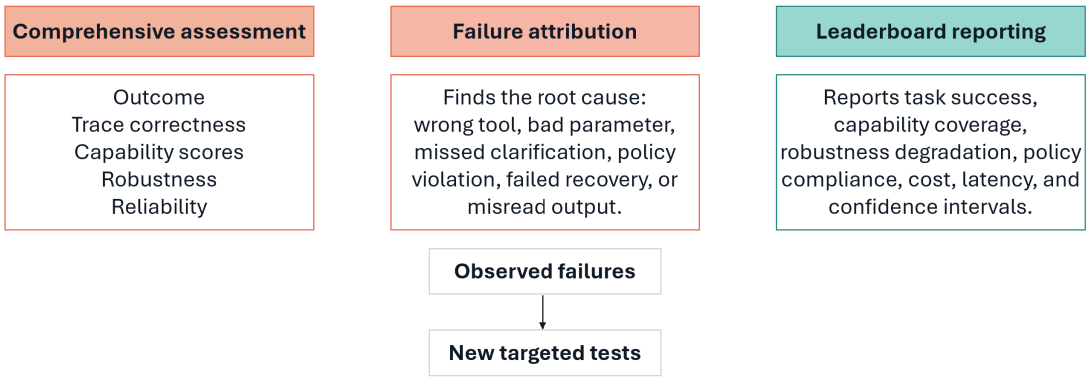

Fig. 15 Assessment pipeline steps 7-9: comprehensive assessment, failure attribution and leaderboard reporting.

policy-compliant under malicious user instructions or tool outputs. *Fault-recovery performance* measures whether the agent responds appropriately to execution failures. Finally, *dynamic capability coverage* measures whether the agent maintains adequate performance for each capability across the original seed tasks and their generated variants.

Failure Attribution. The eighth stage performs failure attribution. When a run fails, the framework identifies the root capability, root step, and error propagation path. For example, a wrong final state may be attributed to an incorrect parameter value, which then propagates through a valid API call to an incorrect final response. Failures can also originate from missing clarification, wrong tool selection, dependency violations, policy violations, or misinterpretation of API outputs. The attribution results are used not only for reporting, but also to drive adaptive test generation. Capabilities with high failure rates or low coverage can be targeted with additional paraphrased, perturbed, or fault-injected test cases, creating a feedback loop in which observed failures generate harder and more diagnostic evaluations.

This feedback loop can be implemented using coverage-guided and failure-guided test generation. Coverage-guided generation prioritizes capabilities, APIs, parameter combinations, and environment conditions that are underrepresented in the current benchmark. Failure-guided generation prioritizes mutations around observed weaknesses, such as a specific tool-selection error, a recurring parameter-format mistake, or a repeated failure to reject malicious tool output. Together, these mechanisms allow the benchmark to expand in directions that are empirically useful, rather than generating variants uniformly at random.

Leaderboard Reporting. The final stage aggregates the assessment results into leaderboard metrics. The leaderboard will report both aggregate and diagnostic measures, including task success rate, trace correctness, capability scores, capability coverage, robustness under dynamic variants, reliability across repeated runs, efficiency, policy compliance, and failure profiles. Capability coverage is particularly important because two agents may achieve similar task success while demonstrating different breadths of competence. We define capability coverage as the proportion of capability categories for which an agent achieves a score above a predefined threshold. A dynamic capability coverage score can also be reported by requiring the agent to remain above the threshold across original, paraphrased, perturbed, adversarial, and

fault-injected variants. This ensures that leaderboard rankings reflect both capability breadth and robustness, rather than only success on a static test set.

The leaderboard will also report benchmark-validity and robustness metadata. This includes the number of seed tasks, number of generated variants, mutation operators used, valid mutant yield, capability coverage, API-parameter coverage, robustness degradation, confidence intervals, and the performance of trivial or baseline agents. Reporting these quantities makes the leaderboard more interpretable and reduces the risk that a single aggregate score conceals benchmark weaknesses, evaluation shortcuts, or narrow overfitting to a static benchmark.

Overall, the proposed pipeline turns the framework from a static evaluation protocol into a dynamic and diagnostic assessment process. It begins with a capability taxonomy, constructs seed benchmarks, expands them through controlled transformations, perturbations, fuzzing-inspired mutations, and environment faults, validates their quality, executes agents in sandboxed environments, analyses traces, attributes failures, and aggregates results into interpretable leaderboard metrics. This design preserves the central principle of the framework, where API-calling agents should be evaluated by the capabilities they reliably demonstrate under realistic and variable operating conditions, rather than by final task success alone.

7 Benchmark and Dataset Collection for Assessment

As part of Milestone 1, we constructed and curated a structured collection of existing benchmarks and datasets related to API-calling agents, tool-use systems, agentic workflows, and closely related evaluation settings. The benchmark collection and literature analysis presented in this report were used to identify existing benchmark construction practices, evaluation methodologies, capability coverage, and current limitations of API-calling benchmark ecosystems.

The collection is maintained as a structured benchmark catalogue accompanied by benchmark-level meta-data describing multiple characteristics of each benchmark and dataset. The collected meta-data includes, among others, benchmark type, domain, task type, tool/API setting, input/output characteristics, evaluation methods, metrics, dataset/source information, references, and links.

The benchmark collection intentionally adopts a broad scope. In addition to benchmarks directly targeting API-calling and tool-use evaluation, it also includes benchmarks related to planning, ambiguity handling, reasoning, workflow execution, and environment-grounded agents. Some code-generation benchmarks involving tool interactions are also included when relevant. More generally, benchmarks were included whenever they provided useful insights for capability analysis, benchmark construction, or assessment methodologies.

The collected benchmark catalogue served as an important foundation for the capability analysis and benchmark survey conducted in this report. In particular, it informed the identification of capability dimensions and assessment requirements discussed in Section 5, as well as the analysis of benchmark construction and assessment techniques summarized in Section 6.

The collection will be incrementally extended throughout the project as new benchmarks, datasets, and evaluation frameworks become available. In subsequent milestones, it will support multiple activities including: (i) the identification of benchmark gaps and capability coverage limitations; (ii) the construction and augmentation of new benchmark instances; (iii) the

derivation of benchmark generation strategies and quality-control mechanisms; and (iv) the design and validation of the proposed capability-based assessment framework.

More specifically, the benchmark collection will directly support the capability-based benchmark construction pipeline described in Section 5 and the capability-based assessment framework introduced in Section 6. The collection will also provide a reusable resource for comparative analysis, capability-oriented evaluation, and future benchmark curation activities conducted during the project.

The benchmark and dataset collection is available through the project GitHub site. An extract containing representative benchmark examples and associated meta-data is also provided in Appendix A.

8 LLM-based Pipeline for API Taxonomy Construction from API-Calling Benchmarks (Preliminary Study)

This task in the project investigates the automatic construction of a taxonomy of APIs using high-abstraction semantic representations generated by LLMs. The objective is to classify API methods into semantically meaningful categories that can support agentic AI systems in reasoning about tools and services at a higher conceptual level.

As part of this work, we implemented a pipeline that will support Milestone 2 of the project, particularly the construction and organization of API-calling benchmarks and action knowledge resources. The proposed pipeline leverages API-calling agent benchmarks as seed sources for constructing the taxonomy. It first generates compact semantic compressions (2–5 words) for API methods using LLMs, while handling duplicate functionalities through similarity-based grouping and LLM-assisted consolidation. It then applies a novel iterative clustering approach that constructs semantic clusters, assigns noisy points using LLM reasoning, and progressively refines clusters into hierarchical taxonomy layers. The approach was evaluated using benchmarks including ToolBench [5], ToolAlpaca [114], MCP-Bench [76], MCP-Universe [87], MCP-AgentBench [77], API-Bank [55], TaskBench [107], and SealTools [115], with additional datasets such as APIBlend [51] and BFCL [10] planned for integration.

Preliminary results show that the proposed clustering strategy outperforms standard approaches such as UMAP + HDBSCAN on this highly inseparable data. The work also explored multiple embedding spaces and alternative semantic-compression strategies to improve clustering quality and scalability.

9 Project Set Up and Coordination

Since the start of the project, the team has been working on the planned milestones and deliverables according to the project schedule. To support project coordination and collaboration, we established regular weekly internal meetings involving all DCU project team members. We also established an organizational coordination structure involving both the DCU and TII teams, including the identification of key contacts, project responsibilities, and coordination roles across the two organizations. In addition, we established regular coordination meetings with TII, typically held every two weeks, although during some periods they are organized monthly depending on project activities and milestones. These meetings are used to discuss

progress updates, technical presentations, deliverables, benchmarking activities, and upcoming project tasks.

As part of the project infrastructure, we established a GitHub organization and project website to centralize and disseminate project resources and outcomes. These resources will be populated gradually as information becomes available throughout the project. The repository structure includes: (i) publications and related references; (ii) a curated list of API-calling and tool-calling LLM agent benchmarks; (iii) project outcomes including reports, presentations, demonstrations, and publications; (iv) code repositories for the different project components, including benchmark construction, assessment pipelines, and LLM-agent middleware components; (v) related tools and external resources. Initially, the website will be populated with related publications and references, identified benchmarks, the Milestone 1 project report, and related presentations.

The project will also release benchmark datasets, generated tasks, execution traces, and related resources through both the GitHub organization and Hugging Face repositories to support reproducibility, benchmarking, and community adoption.

10 Summary and Next Milestones

This section summarizes the main outcomes of Milestone 1 and outlines the next phases of the project. It first summarizes the capability-based assessment foundations introduced in this report, then presents the planned benchmark construction and assessment pipelines that will be developed in subsequent milestones. Finally, it introduces the proposed LLM-agent middleware architecture and its main collaborating agents, which will support robust, adaptive, and quality-aware API-calling systems.

10.1 Summary of the API-Calling Agent Benchmark Framework (Milestone 1)[1]

This report presented the proposed benchmark framework for API-calling LLM agents (Milestone 1), with a particular focus on the challenges of systematically evaluating their behaviour, reliability, and robustness in realistic environments. While the report focused on API-calling agents, the identified capability space and assessment principles provide a foundation that will be extended in future milestones toward more advanced middleware and multi-agent settings, including capabilities related to context management, memory, long-term interaction, orchestration, and coordination across agents and services.

First, the report introduced a capability-based assessment framework for API-calling agents, grounded in the analysis of existing literature and benchmarks. Rather than evaluating agents solely based on final task success, the framework characterizes agent behaviour through a structured capability space organized across three main dimensions: user input understanding, agent reasoning, and API-based action execution. Within these dimensions, the report identified fine-grained attributes capturing critical aspects of agent behaviour, including task complexity, domain and linguistic diversity, ambiguity and error handling, API selection, parameter grounding, dependency and ordering constraints, policy compliance, execution robustness, and

[1] Milestones are described in the project Gantt-Chart.

response interpretation. Together, these dimensions provide a comprehensive foundation for assessing the reliability and robustness of API-calling agents in realistic environments.

Second, the report analysed existing benchmark construction techniques that will inform the benchmark construction pipeline developed in Milestones 2–3. The analysis highlighted that benchmark construction can rely on multiple types of source data, including API documentation, existing benchmarks, API dependency graphs, and seed task–API call pairs. It also identified several generation approaches, including generation from API documentation, repurposing existing datasets, persona-based simulation, and paraphrasing, together with quality-control mechanisms such as automatic validation, LLM-as-judge approaches, human evaluation, filtering, and correction. These techniques provide the foundations for constructing more realistic and capability-oriented benchmarks.

Third, the report analysed assessment techniques and metrics for evaluating API-calling agents in a capability-based manner. The proposed assessment framework combines outcome-based, trace-based, capability-based, and environment-aware evaluation, while treating reliability as a repeated-run property rather than a single execution result. It also identified complementary assessment methods including focused test cases, composite benchmarks, paired perturbation tests, sandboxed execution, trace analysis, checklist-based review, preference-based evaluation, and verifier assessment. These methods are associated with metrics covering task success, tool-use correctness, parameter accuracy, sequence validity, execution success, robustness, safety and policy compliance, efficiency, and reliability.

10.2 Capability-Based Benchmark Construction Pipeline (Milestones 2–3)

In Milestones 2 and 3, the project will propose and implement a capability-based benchmark construction pipeline for API-calling agents that integrates data curation, automated task generation, dataset augmentation, and multi-stage quality-control mechanisms. The pipeline will combine curated task instances from existing API-calling and cross-domain benchmarks with newly generated tasks derived from API documentation, dependency graphs, tool-use demonstrations, and crowd-sourced inputs. This will enable the scalable construction of simple, composite, and nested API-calling tasks while broadening domain and workflow coverage.

The construction process will be guided by the capability dimensions introduced in this report, enabling benchmark instances to target specific capabilities such as task understanding, reasoning, planning, API selection, parameter grounding, robustness, recovery, and policy compliance. The resulting benchmark collection will therefore support both focused capability-oriented evaluation and complex multi-capability workflows.

The pipeline will also investigate techniques for extracting, transforming, and organizing pairs of natural language utterances and executable actions, including action sequences, parameters, and execution constraints. This will support the creation of structured action knowledge resources spanning multiple domains and levels of granularity, from atomic actions to high-level composite workflows. Relationships between actions, parameters, dependencies, and execution constraints will be derived from API descriptions, demonstrations, execution traces, and workflow structures.

To increase benchmark diversity and realism, the curated datasets will be automatically augmented using LLM-based paraphrasing, perturbation, and task synthesis techniques. The

augmentation process will generate lexically, syntactically, and semantically diverse utterances, alternative workflow formulations, noisy and ambiguous inputs, adversarial instructions, and additional execution constraints. The pipeline will also investigate persona-conditioned generation and diversity-aware augmentation strategies to simulate different user behaviours, communication styles, expertise levels, and organizational contexts. Particular attention will be given to generating utterances that uniformly cover API parameters, optional arguments, constraints, and varying workflow complexities.

The generated and curated benchmark instances will be filtered through a combination of automatic validation, execution-based verification, quality filtering, and LLM-as-judge assessment to ensure correctness, consistency, naturalness, and adherence to API specifications and workflow constraints. The project will also investigate constraint-level validation and consistency-checking techniques to assess whether generated tasks and responses satisfy individual execution requirements under different perturbation and prompting settings. In addition, metrics and validation techniques will be investigated for continuously assessing benchmark quality, including coverage, diversity, and reliability across domains and task types. A final validation and curation stage will maintain benchmark quality and reliability before inclusion in the final benchmark suite.

The resulting benchmark collection will support the evaluation of multiple API-calling capabilities, including task understanding, reasoning, planning, API selection, parameter grounding, robustness to noisy and adversarial inputs, recovery from execution failures, and the execution of complex multi-step and nested API workflows.

10.3 Capability-Based Assessment Pipeline (Milestone 4)

In Milestone 4, the project will propose and implement a capability-based assessment pipeline for API-calling systems that integrates dynamic test generation, sandboxed execution, trace analysis, failure attribution, and leaderboard reporting. The objective is to evaluate systems not only according to final task success, but also according to the capabilities demonstrated during execution, including intent understanding, reasoning, API selection, parameter grounding, robustness, recovery, and policy compliance.

The assessment process will begin with the definition of a capability taxonomy derived from the dimensions introduced in this report. Seed benchmark instances produced by the benchmark construction pipeline will then be associated with capability labels to support capability-level evaluation and diagnostic analysis. These benchmarks will cover both focused tasks targeting individual capabilities and composite workflows requiring multiple interacting capabilities.

To support robustness-oriented evaluation, the assessment pipeline will generate dynamic test cases using paraphrasing, perturbation, adversarial mutation, and environment mutation techniques. These transformations will introduce linguistic variation, noisy and ambiguous inputs, malicious instructions, and operational failures such as API timeouts, schema changes, or malformed outputs. Both seed and dynamically generated tasks will undergo validation to ensure solvability, correctness, and consistency with the target assessment capabilities.

Agents and LLMs will then be executed within sandboxed environments that capture execution traces, API interactions, recovery actions, and final environment states. The collected traces will support multi-layer assessment, including outcome evaluation, trace-level analysis, capability assessment, robustness evaluation across dynamic variants, and reliability analysis

across repeated runs. Aggregated results will be used to report leaderboard metrics such as task success, capability coverage, robustness, reliability, efficiency, policy compliance, and failure profiles, providing a more realistic and diagnostic evaluation of API-calling systems.

10.4 LLM-Agent Middleware for API-Calling Systems (Milestones 5–7)

In Milestones 5–7, the project will propose and implement a modular middleware composed of collaborating LLM-based agents and components that improve the robustness, adaptability, and maintainability of tool-using systems. Rather than relying on monolithic orchestration frameworks, the proposed middleware will support capabilities such as composability reasoning, orchestration of tools and services, memory and state management, automated onboarding of new tools, reflection over previous executions, and recovery from execution and communication breakdowns.

The middleware will integrate specialized agents for task decomposition and orchestration, action grounding, memory and context management, reflection and synthesis of reusable workflows, and breakdown recovery. Building on the assessment foundations introduced in this report, the middleware will also integrate quality-aware capabilities for execution monitoring, trace analysis, robustness testing, policy-compliance verification, hallucination detection, and recovery-oriented evaluation under noisy, adversarial, and fault-injected conditions.

10.4.1 State and Memory Agent (SMA)

The middleware will include a self-adaptive State and Memory Agent (SMA) that continuously learns from user-agent interactions, tool-use demonstrations, execution trajectories, failures, corrections, and recovery attempts. Rather than simply storing traces, SMA transforms execution experience into reusable contextual knowledge that can improve future task execution.

SMA will be especially important when tool documentation is incomplete, inconsistent, outdated, or unavailable. In such situations, the agent will refine the middleware's understanding of tool functionality, parameter schemas, constraints, expected outputs, and usage patterns from demonstrations and accumulated execution experience. It will also capture knowledge acquired through corrections, evaluator feedback, and failed trajectories, including why a tool call, plan, or parameter choice was incorrect and which corrections were effective.

At inference time, SMA will retrieve and organize relevant experience to provide high-quality context for downstream agents. This context may include similar trajectories, successful tool-use patterns, previously observed execution outcomes, applicable corrections, organizational constraints, and user-specific preferences relevant to the current task. In this way, SMA supports more grounded planning, better tool selection, improved parameter grounding, and reduced repetition of previously observed mistakes.

10.4.2 Breakdown Recovery Agent (BRA)

The middleware will include a Breakdown Recovery Agent (BRA) responsible for detecting, diagnosing, and recovering from failures occurring during user-agent interaction and tool-based execution. These failures may include ambiguous or incomplete user requests, incorrect intent

inference, invalid tool selection, premature tool invocation, parameter errors, misinterpretation of tool outputs, policy violations, execution failures, and deviations from the user's objective.

BRA will build on the capability-based assessment framework introduced in this report by using execution traces and agent trajectories to identify where and why a breakdown occurs. Rather than treating recovery as an ad hoc repair step, BRA will maintain structured mappings between failure types, trajectory patterns, execution-rule violations, and recovery strategies. Inspired by recent trace-based assessment and recovery approaches, BRA will analyse intermediate reasoning steps, tool interactions, workflow transitions, execution side effects, and compliance with prompt- and policy-level behavioural constraints, rather than relying solely on final task outcomes. The agent will also investigate log-based and compensation-oriented recovery mechanisms that maintain structured execution histories and recovery actions to support reliable rollback, retry, redirection, and safe compensation of failed or partially completed workflows while minimizing unnecessary recomputation and execution overhead. Recovery actions may include clarification questions, reformulation of task context, correction of tool-use decisions, user confirmation requests, retry strategies, compensation actions, or trajectory redirection.

At inference time, BRA will use the current conversation state, tool execution context, and memory retrieved from SMA to provide targeted interventions and feedback to task-executing agents. Over time, BRA will continuously adapt its recovery strategies using observed failures, successful recovery trajectories, corrections, evaluator feedback, and user feedback, enabling systematic recovery from breakdowns and improving robustness across repeated interactions.

At inference time, BRA will use the current conversation state, tool execution context, and memory retrieved from SMA to provide targeted interventions and feedback to task-executing agents. Over time, BRA will continuously adapt its recovery strategies using observed failures, successful recovery trajectories, corrections, evaluator feedback, and user feedback, enabling systematic recovery from breakdowns and improving robustness across repeated interactions.

10.4.3 Reflection and Insight Agent (RIA)

The middleware will include a Reflection and Insight Agent (RIA) responsible for analysing historical conversations, execution trajectories, user and expert feedback, corrections, and lessons learned in order to discover reusable workflow knowledge and higher-level execution insights.

While SMA maintains contextual memory for inference-time retrieval, RIA reflects over accumulated experience to identify recurring task patterns, co-occurring actions, repeated corrections, inefficient execution paths, and successful recovery behaviours. Rather than reusing full trajectories or generating only abstract guidelines, RIA synthesizes reusable intermediate structures such as workflow templates, composite actions, tool-chain patterns, execution rules, and best-practice guidance.

For example, repeated trajectories involving event lookup, participant extraction, policy checking, and notification steps may be generalized into reusable workflow structures, while task-specific details remain dynamically resolved at runtime. RIA will also identify wasteful trajectories, redundant tool use, repeated mistakes, and behaviours conflicting with organizational rules or user expectations. The resulting insights and reusable workflows can then be incorporated into the Action Knowledge Graph and reused by planning, orchestration, and recovery agents.

In this way, RIA transforms accumulated execution experience into reusable capabilities, execution guidance, and improvement rules that help the middleware become progressively more adaptive, efficient, and reliable over time.

10.4.4 Composer and Orchestrator Agent (COA)

The middleware will include a Composer and Orchestrator Agent (COA) responsible for decomposing complex user tasks into executable workflows and dynamically orchestrating tools, services, and specialized agents or models. Rather than relying on static orchestration topologies, the COA continuously adapts orchestration decisions according to the evolving task state, execution context, user constraints, and feedback from previous executions.

Users will express goals, constraints, and preferences through natural language instructions. Before generating a plan, the COA derives task-specific acceptance criteria from user requests, organizational policies, contextual information, execution constraints, and quality-of-service requirements. The COA only generates and executes workflows that satisfy these criteria while avoiding hallucinated, incomplete, unsafe, or non-grounded actions.

The COA is powered by LLMs and the Action Knowledge Graph (AKG), which maintains executable actions together with their metadata, constraints, dependencies, applicability conditions, demonstrations, reusable workflow structures, and embeddings. The COA formulates orchestration as a sequential reasoning and decision-making process where the orchestrator dynamically selects which tools, agents, or models should be invoked at each stage of execution.

Inspired by recent orchestration approaches, the COA treats intelligence as emerging from the coordination of heterogeneous tools and agents rather than from a single monolithic model. Depending on the task complexity, execution cost, and user preferences, the orchestrator may delegate subtasks to specialized agents or lightweight models instead of systematically relying on the most powerful model available. This enables cost-aware and preference-aware orchestration policies that balance execution quality, latency, and computational efficiency.

The COA also leverages contextual knowledge and execution experience maintained by SMA, including relevant trajectories, corrections, organizational constraints, user preferences, and prior successful or failed executions. In addition, the COA reuses orchestration strategies, workflow structures, and reusable execution patterns synthesized by RIA. Over time, orchestration policies continuously evolve from execution feedback and trajectory analysis to prioritize effective coordination patterns, suppress inefficient reasoning paths, and improve robustness and efficiency across tasks and domains.

References

[1] Minaee, S., Mikolov, T., Nikzad, N., Chenaghlu, M., Socher, R., Amatriain, X., Gao, J.: Large language models: A survey. arXiv preprint arXiv:2402.06196 (2024)

[2] Huang, X., Liu, W., Chen, X., Wang, X., Wang, H., Lian, D., Wang, Y., Tang, R., Chen, E.: Understanding the planning of llm agents: A survey. arXiv preprint arXiv:2402.02716 (2024)

[3] Lu, P., Peng, B., Cheng, H., Galley, M., Chang, K.-W., Wu, Y.N., Zhu, S.-C., Gao, J.: Chameleon: Plug-and-play compositional reasoning with large language models. In: Oh,

A., Naumann, T., Globerson, A., Saenko, K., Hardt, M., Levine, S. (eds.) Advances in Neural Information Processing Systems, vol. 36, pp. 43447–43478. Curran Associates, Inc., Red Hook, New York. (2023)

[4] Schick, T., Dwivedi-Yu, J., Dessì, R., Raileanu, R., Lomeli, M., Hambro, E., Zettlemoyer, L., Cancedda, N., Scialom, T.: Toolformer: Language models can teach themselves to use tools. Advances in neural information processing systems **36**, 68539–68551 (2023)

[5] Qin, Y., Liang, S., Ye, Y., Zhu, K., Yan, L., Lu, Y., Lin, Y., Cong, X., Tang, X., Qian, B., et al.: Toolllm: Facilitating large language models to master 16000+ real-world apis. arXiv preprint arXiv:2307.16789 (2023)

[6] Patil, S.G., Zhang, T., Wang, X., Gonzalez, J.E.: Gorilla: Large language model connected with massive apis. Advances in Neural Information Processing Systems **37**, 126544–126565 (2024)

[7] Shen, Y., Song, K., Tan, X., Li, D., Lu, W., Zhuang, Y.: HuggingGPT: Solving ai tasks with chatgpt and its friends in hugging face. In: Proceedings of the 37th International Conference on Neural Information Processing Systems, vol. 36. Curran Associates Inc., Red Hook, New York. (2024)

[8] Wei, J., Wang, X., Schuurmans, D., Bosma, M., ichter, b., Xia, F., Chi, E., Le, Q.V., Zhou, D.: Chain-of-thought prompting elicits reasoning in large language models. In: Proceedings of the 36th International Conference on Neural Information Processing Systems, vol. 35, pp. 24824–24837. Curran Associates, Red Hook, New York. (2022)

[9] Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., Cao, Y.: React: Synergizing reasoning and acting in language models. arXiv preprint arXiv:2210.03629 (2022)

[10] Patil, S.G., Mao, H., Yan, F., Ji, C.C.-J., Suresh, V., Stoica, I., Gonzalez, J.E.: The berkeley function calling leaderboard (bfcl): From tool use to agentic evaluation of large language models. In: Forty-second International Conference on Machine Learning (2025)

[11] Song, Y., Xiong, W., Zhu, D., Wu, W., Qian, H., Song, M., Huang, H., Li, C., Wang, K., Yao, R., *et al.*: Restgpt: Connecting large language models with real-world restful apis. arXiv (2023) https://doi.org/10.48550/arXiv.2306.06624

[12] Yehudai, A., Eden, L., Li, A., Uziel, G., Zhao, Y., Bar-Haim, R., Cohan, A., Shmueli-Scheuer, M.: Survey on evaluation of llm-based agents. arXiv preprint arXiv:2503.16416 (2025)

[13] Bandi, A., Kongari, B., Naguru, R., Pasnoor, S., Vilipala, S.V.: The rise of agentic ai: A review of definitions, frameworks, architectures, applications, evaluation metrics, and challenges. Future Internet **17**(9), 404 (2025)

[14] Mohammadi, M., Li, Y., Lo, J., Yip, W.: Evaluation and benchmarking of llm agents: A survey. In: Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery

and Data Mining V. 2, pp. 6129–6139 (2025)

[15] Yao, S., Shinn, N., Razavi, P., Narasimhan, K.: τ-bench: A benchmark for tool-agent-user interaction in real-world domains. arXiv preprint arXiv:2406.12045 (2024)

[16] Gao, X., Xie, S., Zhai, J., Ma, S., Shen, C.: Mcp-radar: A multi-dimensional benchmark for evaluating tool use capabilities in large language models. arXiv preprint arXiv:2505.16700 (2025)

[17] Guo, Z., Xu, B., Zhu, C., Hong, W., Wang, X., Mao, Z.: Mcp-agentbench: Evaluating real-world language agent performance with mcp-mediated tools. In: Proceedings of the AAAI Conference on Artificial Intelligence, vol. 40, pp. 30888–30896 (2026)

[18] Wang, W., Juluan, S., Ling, Z., Chan, Y.-K., Wang, C., Lee, C., Yuan, Y., Huang, J.-t., Jiao, W., Lyu, M.R.: Learning to ask: When LLM agents meet unclear instruction. In: Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing, pp. 21773–21784. Association for Computational Linguistics, Suzhou, China (2025). https://doi.org/10.18653/v1/2025.emnlp-main.1104 . https://aclanthology.org/2025.emnlp-main.1104/

[19] Liang, P., Bommasani, R., Lee, T., Tsipras, D., Soylu, D., Yasunaga, M., Zhang, Y., Narayanan, D., Wu, Y., Kumar, A., et al.: Holistic evaluation of language models. arXiv preprint arXiv:2211.09110 (2022)

[20] Srivastava, A., Rastogi, A., Rao, A., Shoeb, A.A.M., Abid, A., Fisch, A., Brown, A.R., Santoro, A., Gupta, A., Garriga-Alonso, A., et al.: Beyond the imitation game: Quantifying and extrapolating the capabilities of language models. Transactions on machine learning research (2023)

[21] Dubois, Y., Galambosi, B., Liang, P., Hashimoto, T.B.: Length-controlled alpacaeval: A simple way to debias automatic evaluators. arXiv preprint arXiv:2404.04475 (2024)

[22] Wang, Y., Ma, X., Zhang, G., Ni, Y., Chandra, A., Guo, S., Ren, W., Arulraj, A., He, X., Jiang, Z., *et al.*: Mmlu-pro: A more robust and challenging multi-task language understanding benchmark. Advances in Neural Information Processing Systems **37**, 95266–95290 (2024)

[23] Jain, N., Gu, A., Li, W.-D., Yan, F., Zhang, T., Wang, S., Solar-Lezama, A., Sen, K., Stoica, I.: Livecodebench: Holistic and contamination free evaluation of large language models for code. In: International Conference on Learning Representations, vol. 2025, pp. 58791–58831 (2025)

[24] Rein, D., Hou, B.L., Stickland, A.C., Petty, J., Pang, R.Y., Dirani, J., Michael, J., Bowman, S.R.: Gpqa: A graduate-level google-proof q&a benchmark. arXiv preprint arXiv:2311.12022 (2023)

[25] Yue, X., Ni, Y., Zhang, K., Zheng, T., Liu, R., Zhang, G., Stevens, S., Jiang, D., Ren,

W., Sun, Y., *et al.*: Mmmu: A massive multi-discipline multimodal understanding and reasoning benchmark for expert agi. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, pp. 9556–9567 (2024)

[26] Yu, T., Zhang, R., Yang, K., Yasunaga, M., Wang, D., Li, Z., Ma, J., Li, I., Yao, Q., Roman, S., *et al.*: Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-sql task. In: Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing, pp. 3911–3921 (2018)

[27] Li, J., Hui, B., Qu, G., Yang, J., Li, B., Li, B., Wang, B., Qin, B., Geng, R., Huo, N., *et al.*: Can llm already serve as a database interface? a big bench for large-scale database grounded text-to-sqls. Advances in Neural Information Processing Systems **36**, 42330–42357 (2023)

[28] Jimenez, C.E., Yang, J., Wettig, A., Yao, S., Pei, K., Press, O., Narasimhan, K.: Swe-bench: Can language models resolve real-world github issues? In: International Conference on Learning Representations, vol. 2024, pp. 54107–54157 (2024)

[29] Mündler, N., Müller, M.N., He, J., Vechev, M.: Swt-bench: Testing and validating real-world bug-fixes with code agents. Advances in Neural Information Processing Systems **37**, 81857–81887 (2024)

[30] Lu, S., Guo, D., Ren, S., Huang, J., Svyatkovskiy, A., Blanco, A., Clement, C., Drain, D., Jiang, D., Tang, D., et al.: Codexglue: A machine learning benchmark dataset for code understanding and generation. arXiv preprint arXiv:2102.04664 (2021)

[31] Yang, J., Jimenez, C.E., Zhang, A.L., Lieret, K., Yang, J., Wu, X., Press, O., Muennighoff, N., Synnaeve, G., Narasimhan, K.R., et al.: Swe-bench multimodal: Do ai systems generalize to visual software domains? arXiv preprint arXiv:2410.03859 (2024)

[32] Jain, N., Singh, J., Shetty, M., Zheng, L., Sen, K., Stoica, I.: R2e-gym: Procedural environments and hybrid verifiers for scaling open-weights swe agents. arXiv preprint arXiv:2504.07164 (2025)

[33] Niu, C., Li, C., Ng, V., Luo, B.: Crosscodebench: Benchmarking cross-task generalization of source code models. In: 2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE), pp. 537–549 (2023). IEEE

[34] Deng, X., Gu, Y., Zheng, B., Chen, S., Stevens, S., Wang, B., Sun, H., Su, Y.: Mind2web: Towards a generalist agent for the web. Advances in Neural Information Processing Systems **36**, 28091–28114 (2023)

[35] Zhou, S., Xu, F.F., Zhu, H., Zhou, X., Lo, R., Sridhar, A., Cheng, X., Ou, T., Bisk, Y., Fried, D., *et al.*: Webarena: A realistic web environment for building autonomous agents. In: International Conference on Learning Representations, vol. 2024, pp. 15585–15606 (2024)

[36] Koh, J.Y., Lo, R., Jang, L., Duvvur, V., Lim, M., Huang, P.-Y., Neubig, G., Zhou, S., Salakhutdinov, R., Fried, D.: Visualwebarena: Evaluating multimodal agents on realistic visual web tasks. In: Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pp. 881–905 (2024)

[37] Drouin, A., Gasse, M., Caccia, M., Laradji, I.H., Del Verme, M., Marty, T., Boisvert, L., Thakkar, M., Cappart, Q., Vazquez, D., et al.: Workarena: How capable are web agents at solving common knowledge work tasks? arXiv preprint arXiv:2403.07718 (2024)

[38] Boisvert, L., Thakkar, M., Gasse, M., Caccia, M., De Chezelles, T.L., Cappart, Q., Chapados, N., Lacoste, A., Drouin, A.: Workarena++: Towards compositional planning and reasoning-based common knowledge work tasks. Advances in Neural Information Processing Systems **37**, 5996–6051 (2024)

[39] Chezelles, D., Le Sellier, T., Shayegan, S.O., Jang, L.K., Lù, X.H., Yoran, O., Kong, D., Xu, F.F., Reddy, S., Cappart, Q., et al.: The browsergym ecosystem for web agent research. arXiv preprint arXiv:2412.05467 (2024)

[40] Xie, T., Zhang, D., Chen, J., Li, X., Zhao, S., Cao, R., Hua, T.J., Cheng, Z., Shin, D., Lei, F., *et al.*: Osworld: Benchmarking multimodal agents for open-ended tasks in real computer environments. Advances in Neural Information Processing Systems **37**, 52040–52094 (2024)

[41] Bonatti, R., Zhao, D., Bonacci, F., Dupont, D., Abdali, S., Li, Y., Lu, Y., Wagle, J., Koishida, K., Bucker, A., et al.: Windows agent arena: Evaluating multi-modal os agents at scale. arXiv preprint arXiv:2409.08264 (2024)

[42] Zhang, C., Yang, Z., Liu, J., Li, Y., Han, Y., Chen, X., Huang, Z., Fu, B., Yu, G.: Appagent: Multimodal agents as smartphone users. In: Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems, pp. 1–20 (2025)

[43] Xu, F.F., Song, Y., Li, B., Tang, Y., Jain, K., Bao, M., Wang, Z., Zhou, X., Guo, Z., Cao, M., et al.: Theagentcompany: benchmarking llm agents on consequential real world tasks. Advances in Neural Information Processing Systems **38** (2026)

[44] Yan, Y., Wang, S., Du, J., Yang, Y., Shan, Y., Qiu, Q., Jia, X., Wang, X., Yuan, X., Han, X., et al.: Mcpworld: A unified benchmarking testbed for api, gui, and hybrid computer use agents. arXiv preprint arXiv:2506.07672 (2025)

[45] Chen, Z., Chen, S., Ning, Y., Zhang, Q., Wang, B., Yu, B., Li, Y., Liao, Z., Wei, C., Lu, Z., *et al.*: Scienceagentbench: Toward rigorous assessment of language agents for data-driven scientific discovery. In: International Conference on Learning Representations, vol. 2025, pp. 96934–96990 (2025)

[46] Mialon, G., Fourrier, C., Wolf, T., LeCun, Y., Scialom, T.: Gaia: a benchmark for general ai assistants. In: International Conference on Learning Representations, vol. 2024, pp. 9025–9049 (2024)

[47] Ma, C., Zhang, J., Zhu, Z., Yang, C., Yang, Y., Jin, Y., Lan, Z., Kong, L., He, J.: Agentboard: An analytical evaluation board of multi-turn llm agents. Advances in neural information processing systems **37**, 74325–74362 (2024)

[48] Santos, V.G., Benatallah, B., Berro, A., MacMahon, S.T.: Diverse utterances generation with gpt to improve task-oriented chatbots and apis integration. In: 2024 2nd International Conference on Foundation and Large Language Models, pp. 42–50 (2024). https://doi.org/10.1109/FLLM63129.2024.10852454

[49] Moon, S., Jha, S., Erdogan, L.E., Kim, S., Lim, W., Keutzer, K., Gholami, A.: Efficient and scalable estimation of tool representations in vector space. arXiv (2024) https://doi.org/10.48550/arXiv.2409.02141

[50] Chen, Y., Yoon, J., Sachan, D.S., Wang, Q., Cohen-Addad, V., Bateni, M., Lee, C.-Y., Pfister, T.: Re-invoke: Tool invocation rewriting for zero-shot tool retrieval. In: Findings of the Association for Computational Linguistics: EMNLP 2024, pp. 4705–4726. Association for Computational Linguistics, Miami, Florida, USA (2024). https://doi.org/10.18653/v1/2024.findings-emnlp.270

[51] Basu, K., Abdelaziz, I., Chaudhury, S., Dan, S., Crouse, M., Munawar, A., Austel, V., Kumaravel, S., Muthusamy, V., Kapanipathi, P., *et al.*: Api-blend: A comprehensive corpora for training and benchmarking api llms. In: Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pp. 12859–12870 (2024)

[52] Xu, Q., Hong, F., Li, B., Hu, C., Chen, Z., Zhang, J.: On the tool manipulation capability of open-source large language models. arXiv preprint arXiv:2305.16504 (2023)

[53] Lu, J., Holleis, T., Zhang, Y., Aumayer, B., Nan, F., Bai, H., Ma, S., Ma, S., Li, M., Yin, G., *et al.*: Toolsandbox: A stateful, conversational, interactive evaluation benchmark for llm tool use capabilities. In: Findings of the Association for Computational Linguistics: NAACL 2025, pp. 1160–1183 (2025)

[54] Trivedi, H., Khot, T., Hartmann, M., Manku, R., Dong, V., Li, E., Gupta, S., Sabharwal, A., Balasubramanian, N.: Appworld: A controllable world of apps and people for benchmarking interactive coding agents. In: Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pp. 16022–16076 (2024)

[55] Li, M., Zhao, Y., Yu, B., Song, F., Li, H., Yu, H., Li, Z., Huang, F., Li, Y.: Api-bank: A comprehensive benchmark for tool-augmented llms. In: Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing, pp. 3102–3116 (2023)

[56] Basu, K., Abdelaziz, I., Bradford, K., Crouse, M., Kate, K., Kumaravel, S., Goyal, S., Munawar, A., Rizk, Y., Wang, X., *et al.*: Nestful: A benchmark for evaluating llms on nested sequences of api calls. arXiv (2024) https://doi.org/10.48550/arXiv.2409.03797

[57] Han, H., Zhu, T., Zhang, X., Wu, M., Hao, X., Chen, W.: Nestools: A dataset for evaluating nested tool learning abilities of large language models. In: Proceedings of the 31st International Conference on Computational Linguistics, pp. 9824–9844. Association for Computational Linguistics, Eighth Street, Stroudsburg, PA 18360, USA (2025). https://aclanthology.org/2025.coling-main.657/

[58] Tan, H., Zhang, Z., Ma, C., Chen, X., Dai, Q., Dong, Z.: Membench: Towards more comprehensive evaluation on the memory of llm-based agents. In: Findings of the Association for Computational Linguistics: ACL 2025, pp. 19336–19352 (2025)

[59] Du, P.: Memory for Autonomous LLM Agents:Mechanisms, Evaluation, and Emerging Frontiers (2026). https://arxiv.org/abs/2603.07670

[60] He, Z., Wang, Y., Zhi, C., Hu, Y., Chen, T.-P., Yin, L., Chen, Z., Wu, T.A., Ouyang, S., Wang, Z., Pei, J., McAuley, J., Choi, Y., Pentland, A.: MemoryArena: Benchmarking Agent Memory in Interdependent Multi-Session Agentic Tasks (2026). https://arxiv.org/abs/2602.16313

[61] Berro, A., Benatallah, B., Gaci, Y., Benabdeslem, K.: Error types in transformer-based paraphrasing models: A taxonomy, paraphrase annotation model and dataset. In: Joint European Conference on Machine Learning and Knowledge Discovery in Databases, pp. 332–349 (2024). Springer

[62] Pavlick, E., Callison-Burch, C.: Simple ppdb: A paraphrase database for simplification. In: Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers), pp. 143–148 (2016)

[63] Kostić, B., Fallon, C., Risch, J., Löser, A.: Same meaning, different scores: Lexical and syntactic sensitivity in llm evaluation. arXiv preprint arXiv:2602.17316 (2026)

[64] Berro, A., Santos, V., Benatallah, B., Benabdeslem, K.: Llms to replace crowdsourcing in generating syntactically diverse paraphrases for task-oriented chatbots. In: International Conference on Advanced Information Systems Engineering, pp. 145–162 (2025). Springer

[65] Chaves, A.P., Doerry, E., Egbert, J., Gerosa, M.: It's how you say it: Identifying appropriate register for chatbot language design. In: Proceedings of the 7th International Conference on Human-Agent Interaction, pp. 102–109 (2019)

[66] Sheng, Y., Gandhe, S., Kanagal, B., Edmonds, N., Fisher, Z., Tata, S., Selvan, A.: Measuring an llm's proficiency at using apis: a query generation strategy. In: Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, pp. 5680–5689 (2024)

[67] Laban, P., Hayashi, H., Zhou, Y., Neville, J.: Llms get lost in multi-turn conversation. arXiv preprint arXiv:2505.06120 (2025)

[68] Al Sharou, K., Li, Z., Specia, L.: Towards a better understanding of noise in natural

language processing. In: Proceedings of the International Conference on Recent Advances in Natural Language Processing (RANLP 2021), pp. 53–62 (2021)

[69] Zhan, Q., Liang, Z., Ying, Z., Kang, D.: InjecAgent: Benchmarking indirect prompt injections in tool-integrated large language model agents. In: Findings of the Association for Computational Linguistics: ACL 2024, pp. 10471–10506. Association for Computational Linguistics, Bangkok, Thailand (2024). https://doi.org/10.18653/v1/2024.findings-acl.624 . https://aclanthology.org/2024.findings-acl.624/

[70] Chen, Y., Chen, M., Gao, C., Jiang, Z., Li, Z., Ma, Y.: Towards mitigating api hallucination in code generated by llms with hierarchical dependency aware. In: Proceedings of the 33rd ACM International Conference on the Foundations of Software Engineering, pp. 468–479 (2025)

[71] Ponelat, J., Rosenstock, L.: Designing APIs with Swagger and OpenAPI. Simon and Schuster, New York, NY (2022)

[72] Lee, S., Kim, N., Jo, Y.: In-n-out: A parameter-level api graph dataset for tool agents. arXiv preprint arXiv:2509.01560 (2025)

[73] Akshathala, S., Adnan, B., Ramesh, M., Vaidhyanathan, K., Muhammed, B., Parthasarathy, K.: Beyond task completion: An assessment framework for evaluating agentic ai systems. arXiv preprint arXiv:2512.12791 (2025)

[74] Baumann, J., Padmakumar, V., Li, X., Yang, J., Yang, D., Koyejo, S.: Swe-chat: Coding agent interactions from real users in the wild. arXiv preprint arXiv:2604.20779 (2026)

[75] Zhu, Y., Jin, T., Pruksachatkun, Y., Zhang, A., Liu, S., Cui, S., Kapoor, S., Longpre, S., Meng, K., Weiss, R., et al.: Establishing best practices for building rigorous agentic benchmarks. URL https://arxiv. org/abs/2507.02825 (2025)

[76] Wang, Z., Chang, Q., Patel, H., Biju, S., Wu, C.-E., Liu, Q., Ding, A., Rezazadeh, A., Shah, A., Bao, Y., et al.: Mcp-bench: Benchmarking tool-using llm agents with complex real-world tasks via mcp servers. arXiv preprint arXiv:2508.20453 (2025)

[77] Guo, Z., Xu, B., Zhu, C., Hong, W., Wang, X., Mao, Z.: Mcp-agentbench: Evaluating real-world language agent performance with mcp-mediated tools. In: Proceedings of the AAAI Conference on Artificial Intelligence, vol. 40, pp. 30888–30896 (2026)

[78] Misra, D., Islah, N., May, V., Rauby, B., Wang, Z., Gehring, J., Orvieto, A., Chaudhary, M., Muller, E.B., Rish, I., et al.: Gitchameleon 2.0: Evaluating ai code generation against python library version incompatibilities. arXiv preprint arXiv:2507.12367 (2025)

[79] Saeidi, A., Mishra, V., Mukhopadhyay, S., Liu, G., Payani, A., Srinivasa, J., Baral, C.: Fama: Failure-aware meta-agentic framework for open-source llms in interactive tool use environments. arXiv preprint arXiv:2604.25135 (2026)

[80] Guo, D., Wu, J., Yiu, S.M.: Agenteval: Dag-structured step-level evaluation for agentic workflows with error propagation tracking. arXiv preprint arXiv:2604.23581 (2026)

[81] Chen, M., Wang, J., Mu, F., Wang, Y., Liu, Z., Feng, H., Wang, Q.: Seeing the whole elephant: A benchmark for failure attribution in llm-based multi-agent systems. arXiv preprint arXiv:2604.22708 (2026)

[82] Bandi, C., Hertzberg, B., Boo, G., Polakam, T., Da, J., Hassaan, S., Sharma, M., Park, A., Hernandez, E., Rambado, D., et al.: Mcp-atlas: A large-scale benchmark for tool-use competency with real mcp servers. arXiv preprint arXiv:2602.00933 (2026)

[83] Liu, W., Liu, Z., Dai, E., Yu, W., Yu, L., Yang, T., Han, J., Gao, H.: Mcpagentbench: A real-world task benchmark for evaluating llm agent mcp tool use. arXiv preprint arXiv:2512.24565 (2025)

[84] Mitra, K., Zhang, D., Kim, H., Hruschka, E.: Recap: Rewriting conversations for intent understanding in agentic planning. In: Findings of the Association for Computational Linguistics: EACL 2026, pp. 2015–2033 (2026)

[85] Li, H., Mu, C., Chen, J., Ren, S., Cui, Z., Zhang, Y., Bai, L., Hu, S.: Organizing, orchestrating, and benchmarking agent skills at ecosystem scale. arXiv preprint arXiv:2603.02176 (2026)

[86] Patil, S.G., Mao, H., Yan, F., Ji, C.C.-J., Suresh, V., Stoica, I., Gonzalez, J.E.: The berkeley function calling leaderboard (bfcl): From tool use to agentic evaluation of large language models. In: Forty-second International Conference on Machine Learning (2025)

[87] Luo, Z., Shen, Z., Yang, W., Zhao, Z., Jwalapuram, P., Saha, A., Sahoo, D., Savarese, S., Xiong, C., Li, J.: Mcp-universe: Benchmarking large language models with real-world model context protocol servers. arXiv preprint arXiv:2508.14704 (2025)

[88] Xie, J., Zhang, K., Chen, J., Zhu, T., Lou, R., Tian, Y., Xiao, Y., Su, Y.: Travelplanner: a benchmark for real-world planning with language agents. In: Proceedings of the 41st International Conference on Machine Learning. JMLR.org, Miami, USA (2024)

[89] Wu, Q., Liu, W., Luan, J., Wang, B.: ToolPlanner: A tool augmented LLM for multi granularity instructions with path planning and feedback. In: Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing, pp. 18315–18339. Association for Computational Linguistics, Miami, USA (2024). https://doi.org/10.18653/v1/2024.emnlp-main.1018

[90] Yaghoub-Zadeh-Fard, M.-A., Benatallah, B., Casati, F., Barukh, M.C., Zamanirad, S.: User utterance acquisition for training task-oriented bots: a review of challenges, techniques and opportunities. IEEE Internet Computing **24**(3), 30–38 (2020) https://doi.org/10.1109/MIC.2020.2978157

[91] Liu, R., Wei, J., Liu, F., Si, C., Zhang, Y., Rao, J., Zheng, S., Peng, D., Yang, D.,

Zhou, D., et al.: Best practices and lessons learned on synthetic data. arXiv preprint arXiv:2404.07503 (2024)

[92] Liu, Z., Hoang, T., Zhang, J., Zhu, M., Lan, T., Kokane, S., Tan, J., Yao, W., Liu, Z., Feng, Y., *et al.*: APIgen: Automated pipeline for generating verifiable and diverse function-calling datasets. Advances in Neural Information Processing Systems **37**, 54463–54482 (2024)

[93] Qu, C., Dai, S., Wei, X., Cai, H., Wang, S., Yin, D., Xu, J., Wen, J.-R.: Towards completeness-oriented tool retrieval for large language models. In: Proceedings of the 33rd ACM International Conference on Information and Knowledge Management. CIKM '24, pp. 1930–1940. Association for Computing Machinery, New York, NY, USA (2024). https://doi.org/10.1145/3627673.3679847 . https://doi.org/10.1145/3627673.3679847

[94] Budzianowski, P., Wen, T.-H., Tseng, B.-H., Casanueva, I., Ultes, S., Ramadan, O., Gašić, M.: MultiWOZ - a large-scale multi-domain Wizard-of-Oz dataset for task-oriented dialogue modelling. In: Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing, pp. 5016–5026. Association for Computational Linguistics, Brussels, Belgium (2018). https://doi.org/10.18653/v1/D18-1547 . https://aclanthology.org/D18-1547/

[95] Rastogi, A., Zang, X., Sunkara, S., Gupta, R., Khaitan, P.: Towards scalable multi-domain conversational agents: The schema-guided dialogue dataset. In: Proceedings of the AAAI Conference on Artificial Intelligence, vol. 34, pp. 8689–8696 (2020)

[96] Jha, S., Paltenghi, M., Maddila, C., Murali, V., Ugare, S., Chandra, S.: REAP: Automatic Curation of Coding Agent Benchmarks from Interactive Production Usage (2026). https://arxiv.org/abs/2604.01527

[97] Huang, Y., Shi, J., Li, Y., Fan, C., Wu, S., Zhang, Q., Liu, Y., Zhou, P., Wan, Y., Gong, N.Z., *et al.*: Metatool benchmark for large language models: Deciding whether to use tools and which to use. arXiv (2023) https://doi.org/10.48550/arXiv.2310.03128

[98] Mu, H., Xu, Y., Feng, Y., Han, X., Li, Y., Hou, Y., Che, W.: Beyond static evaluation: A dynamic approach to assessing AI assistants' API invocation capabilities. In: Proceedings of the 2024 Joint International Conference on Computational Linguistics, Language Resources and Evaluation, pp. 2342–2353. ELRA and ICCL, ??? (2024). https://aclanthology.org/2024.lrec-main.209/

[99] Elder, B., Murthi, A., Kang, J., Naik, A., Basu, K., Kate, K., Contractor, D.: Live api-bench: 2500+ live apis for testing multi-step tool calling. In: Proceedings of the 19th Conference of the European Chapter of the Association for Computational Linguistics (Volume 1: Long Papers), pp. 3092–3124 (2026)

[100] Truong, K., Fogliato, R., Heidari, H., Wu, S.: Persona-augmented benchmarking: Evaluating llms across diverse writing styles. In: Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing, pp. 22687–22720 (2025)

[101] Ge, T., Chan, X., Wang, X., Yu, D., Mi, H., Yu, D.: Scaling synthetic data creation with 1,000,000,000 personas. arXiv preprint arXiv:2406.20094 (2024)

[102] Chen, Z.-Y., Shen, S., Shen, G., Zhi, G., Chen, X., Lin, Y.: Towards tool use alignment of large language models. In: Al-Onaizan, Y., Bansal, M., Chen, Y.-N. (eds.) Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing, pp. 1382–1400. Association for Computational Linguistics, Miami, Florida, USA (2024). https://doi.org/10.18653/v1/2024.emnlp-main.82 . https://aclanthology.org/2024.emnlp-main.82/

[103] Chen, W., Li, Z., Ma, M.: Octopus: On-device language model for function calling of software APIs. In: Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies, pp. 329–339. Association for Computational Linguistics, Albuquerque, New Mexico (2025). https://doi.org/10.18653/v1/2025.naacl-industry.27

[104] Wang, H., Wang, R., Xue, B., Xia, H., Cao, J., Liu, Z., Pan, J.Z., Wong, K.-F.: AppBench: Planning of multiple APIs from various APPs for complex user instruction. In: Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing, pp. 15322–15336. Association for Computational Linguistics, Miami, USA (2024). https://doi.org/10.18653/v1/2024.emnlp-main.856

[105] Xu, H., Zhao, R., Wang, J., Chen, H.: RESTful-llama: Connecting user queries to RESTful APIs. In: Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing, pp. 1433–1443. Association for Computational Linguistics, Miami, USA (2024). https://doi.org/10.18653/v1/2024.emnlp-industry.105

[106] Iskander, S., Tolmach, S., Shapira, O., Cohen, N., Karnin, Z.: Quality matters: Evaluating synthetic data for tool-using LLMs. In: Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing, pp. 4958–4976. Association for Computational Linguistics, Miami, USA (2024). https://doi.org/10.18653/v1/2024.emnlp-main.285 . https://aclanthology.org/2024.emnlp-main.285/

[107] Shen, Y., Song, K., Tan, X., Zhang, W., Ren, K., Yuan, S., Lu, W., Li, D., Zhuang, Y.: Taskbench: Benchmarking large language models for task automation. Advances in Neural Information Processing Systems **37**, 4540–4574 (2024)

[108] He, J., Fan, Z., Kuang, S., Xiaoqing, L., Song, K., Zhou, Y., Qiu, X.: Fine: Filtering and improving noisy data elaborately with large language models. In: Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers), pp. 8686–8707 (2025)

[109] Vuddanti, S.V., Shah, A., Chittiprolu, S.K., Song, T., Dev, S., Zhu, K., Chaudhary, M.: Paladin: Self-correcting language model agents to cure tool-failure cases. arXiv preprint arXiv:2509.25238 (2025)

[110] Lee, S., Jang, D., Arya, D., Han, G.-e., Song, I., Kim, S., Kim, S., Lee, S., Hong, S., Sek, S., *et al.*: Telagentbench: A multi-faceted benchmark for evaluating llm-based agents in telecommunications. In: Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing: Industry Track, pp. 1173–1211 (2025)

[111] Chen, J., Wang, Y., Wang, J., Xie, X., Li, S., Wang, Q., Xu, F.: Where did it go wrong? capability-oriented failure attribution for vision-and-language navigation agents. arXiv preprint arXiv:2604.25161 (2026)

[112] Goel, K., Rajani, N.F., Vig, J., Taschdjian, Z., Bansal, M., Ré, C.: Robustness gym: Unifying the nlp evaluation landscape. In: Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies: Demonstrations, pp. 42–55 (2021)

[113] Zhou, Y., Xu, A., Zhou, Y., Singh, J., Gui, J., Joty, S.: Variation in verification: Understanding verification dynamics in large language models. arXiv preprint arXiv:2509.17995 (2025)

[114] Tang, Q., Deng, Z., Lin, H., Han, X., Liang, Q., Cao, B., Sun, L.: Toolalpaca: Generalized tool learning for language models with 3000 simulated cases. arXiv preprint arXiv:2306.05301 (2023)

[115] Wu, M., Zhu, T., Han, H., Tan, C., Zhang, X., Chen, W.: Seal-tools: Self-instruct tool learning dataset for agent tuning and detailed benchmark. In: CCF International Conference on Natural Language Processing and Chinese Computing, pp. 372–384 (2024). Springer

[116] Huang, S., Zhong, W., Lu, J., Zhu, Q., Gao, J., Liu, W., Hou, Y., Zeng, X., Wang, Y., Shang, L., Jiang, X., Xu, R., Liu, Q.: Planning, creation, usage: Benchmarking LLMs for comprehensive tool utilization in real-world complex scenarios. In: Findings of the Association for Computational Linguistics, pp. 4363–4400. Association for Computational Linguistics, Bangkok, Thailand (2024). https://doi.org/10.18653/v1/2024.findings-acl.259

[117] Sarwar, T., Moghimifar, F., Hoang, C.D.V., Ma, X., Xu, S.C., Saleh, F., Zaremoodi, P., Sil, A., Kirchhoff, K.: Clarity: A framework and benchmark for conversational language ambiguity and unanswerability in interactive nl2sql systems. arXiv preprint arXiv:2604.22313 (2026)

Appendix A Benchmark and Dataset Collection

Appendix B Evaluation Metrics

Table A1 Examples of Existing API-calling Benchmarks.

Benchmark	Summary
BFCL [10]	An executable leaderboard benchmark evaluating LLMs on simple, composite, single- and multi-step function calling across multiple programming languages, with instances generated by human experts, crowd workers, and LLMs.
ToolBench [5]	Includes 16,464 real-world APIs scraped from RapidAPI Hub, paired with 126,486 LLM-generated instances covering multiple domains and composite tasks.
ToolAlpaca [114]	Includes 3,938 LLM-generated instances covering 426 real-world APIs across single- and multi-step tasks spanning multiple domains.
NESTful [56]	Includes 300 instances requiring nested API call sequences, where the output of one API call serves as input to subsequent calls, generated through human–LLM collaboration.
APIBench [6]	Includes 16,450 LLM-generated instances focused on machine learning applications, covering single-turn tasks that each involve the use of a single API per task.
APIGen [92]	Includes 60,000 LLM-generated instances spanning 3,673 APIs across multiple domains, produced through a multi-stage quality-control pipeline to ensure correctness and diversity.
API-Bank [55]	Includes 314 manually constructed dialogues involving 753 API calls across diverse domains, designed to assess LLM agents' capabilities in planning, retrieving, and calling APIs.
UltraTool [116]	Includes 5,824 instances combining LLM-based synthetic generation with human refinement across 2,032 APIs, evaluating multiple agent capabilities such as planning, API selection, and API calling.
τ-Bench [15]	A benchmark for evaluating LLM agents in dynamic, multi-turn customer service scenarios, where agents must interact with a simulated user, invoke domain-specific APIs, and comply with policy documents.
NoisyToolBench [18]	Includes 200 manually modified instances built by introducing noise into problem-free tasks from ToolBench [5], evaluating LLM agents' ability to handle ambiguous user requests and decide when to seek clarification.
MCP-Bench [76]	A benchmark connecting LLMs to 28 live MCP servers spanning 250 APIs across domains such as finance, travel, and scientific computing, evaluating multi-step API use and planning under composite user tasks.
MCP-Universe [87]	Includes 231 tasks built around 11 real-world MCP servers across 6 domains, requiring long-horizon reasoning and assessed through rigorous evaluation criteria, including agent format compliance, time-invariant content matching, and dynamic evaluators that retrieve real-time ground truth for temporally sensitive tasks.

Table B2 Summary of assessment families, evaluation focus, representative metrics, and supporting references for API-calling agent evaluation.

Assessment family	Assessment focus	Main metrics / signals	References
Outcome-based task success	Measures whether the agent completes the intended task or produces the correct final answer.	Pass rate, task completion, execution accuracy, task finish score, strict/lenient exact match, claim coverage.	[79, 82, 83, 110, 117]
Tool-use and API execution	Evaluates whether the agent selects the correct tool, constructs valid arguments, executes calls, and handles API outputs.	Tool-selection accuracy, parameter correctness, API-call validity, tool discovery, syntax correctness, parameter coverage, execution success.	[66, 82, 83]
Trace and process evaluation	Assesses the intermediate reasoning and action sequence rather than only the final result.	Step-level quality, trace correctness, node/edge correctness, failure-detection recall, root-cause accuracy, sequence correctness.	[73, 80, 81]
Capability-oriented assessment	Organizes evaluation around specific agent capabilities such as planning, tool use, ambiguity handling, recovery, and instruction following.	Capability score, capability coverage, capability-specific oracle scores, reasoning/planning/action/RAG/IF accuracy.	[66, 110, 111]
Robustness and dynamic testing	Tests whether performance remains stable under paraphrases, perturbations, ambiguity, adversarial inputs, and environment variation.	Robustness slices, perturbation degradation, paraphrase stability, ambiguity/unanswerability performance, adversarial robustness.	[84, 110, 112, 117]
Failure attribution and diagnosis	Identifies where failures occur and which capability, step, or agent component caused them.	Agent-level attribution accuracy, step-level attribution accuracy, root capability, root step, failure clusters, feedback scores.	[79–81, 111]
Reliability and repeated-run behavior	Measures whether agent behavior is consistent across repeated executions or generated candidates.	pass@k, pass^k, repeated-run consistency, trajectory reliability, verification accuracy, true positive/negative rates.	[73, 79, 113]
Efficiency and resource use	Evaluates whether the agent completes tasks with reasonable cost, time, and tool-use overhead.	Time efficiency, token efficiency, latency, cost, redundant calls, user effort, coding efficiency.	[74, 82, 83]
Safety, policy, and trustworthiness	Assesses whether agents comply with rules, avoid unsafe behavior, and whether benchmark scores are trustworthy.	Safety score, policy compliance, guardrail violations, introduced vulnerabilities, task validity, outcome validity, contamination controls, confidence intervals.	[73–75]
Preference and quality assessment	Evaluates open-ended outputs or plans where multiple answers may be acceptable.	Pairwise win rate, Bradley–Terry score, rubric score, LLM-as-judge score, semantic similarity, graph edit distance.	[73, 84, 85]